# Supplemental Material for "Memory–Output Tradeoffs for Merging Dispersed Rotations in Downstream Quantum Compilers"

Jinze Yang[1], Yangyang Li[2], and Xiu-Hao Deng[3]

[1]School of Physics, Xidian University, Xi'an 710071, China
[2]School of Artificial Intelligence, Xidian University, Xi'an 710071, China
[3]Shenzhen International Quantum Academy, Shenzhen 518048, China

## S1 Scope and evidence policy

This document contains expanded experimental protocols, negative results, status accounting, a historical/stress-result registry, and data provenance for the main manuscript, together with material moved out of it: the record codec and field-level cut accounting (Section S9), the certificate domain and its soundness proof (Section S10), the exact-semantics results for fixed-width circuits with full proofs (Section S11), additional experiments and protocols (Section S12), artifact engineering (Section S13), supplementary experiments (Section S14), and reproducibility details (Section S15). The proofs of every result stated in the main text are in the main text or its appendix. Historical files are retained for auditability but are not promoted to canonical W3–W9 evidence unless they appear in `data/manifest.yaml`.

Only `completed_valid` or a campaign-specific `completed` paired with a passing independent certificate is an accepted quality outcome. `timeout`, `memory_error`, `unsupported`, `predicate_error`, `correctness_failure`, and `numerical_inconclusive` remain separate categories. A failed or unsupported cell is never assigned an infinite, zero, or best-available metric.

Table S1: Canonical campaign inventory. Counts are generated from registered frozen sources; status-only rows remain in their source tables.

| Package | Canonical cells | Audit status |
|---|---|---|
| W3 tradeoff traces | 592 | all theorem diagnostics pass |
| W4 matched representations | 1,080 | 1,065 completed; 15 predicate errors |
| W5 external baselines | 1,728 | 1,695 completed; 33 predicate errors |
| W6 certificate audit | 56 mutations | 0 false accepts |
| W7 fixed-error QRE | 288 | all completed-valid |
| W8 natural factorial | 21,060 | all completed-valid |
| W9 normalized index | 24,748 | unique complete keys |

## S2 Instrumented commitment–state–IR campaign

The W3 reference suite is a collection of formal-model objects, not wrappers around production compilers. The event schema records input position, pass, control/store/window bytes, parameter bytes, serialized live-IR bytes, committed output delta and total, RSS, and wall time. The theorem coordinate $S$ is computed from the complete serialized cut state. RSS is an ancillary operational measurement and is never used as a proof surrogate.

The grid crosses $m$, update rounds, $K$, separation margin, forward passes, memory cap, schedule, and seed. Every run has a per-run manifest and event log. The aggregate file `data/frozen/`

Table S2: Reference compiler behaviors. Memory percentages refer to the payload portion of the full aggregate vector; framing remains charged.

| Compiler | Aggregate memory | Commitment policy |
|---|---|---|
| Echo | none | append each update in stream order |
| Full aggregation | full | retain vector, emit canonical aggregate |
| Hybrid-25 | 25% | aggregate public block, pass through remainder |
| Hybrid-50 | 50% | aggregate public block, pass through remainder |
| Hybrid-75 | 75% | aggregate public block, pass through remainder |
| IR materializing | full plus IR | retain explicit materialized stream |

`tradeoff_summary.csv` reports both the exact transcript residual and the state-cap envelope residual. The frozen anomaly table is empty; this is reported as an observed campaign fact, not substituted for the theorem.

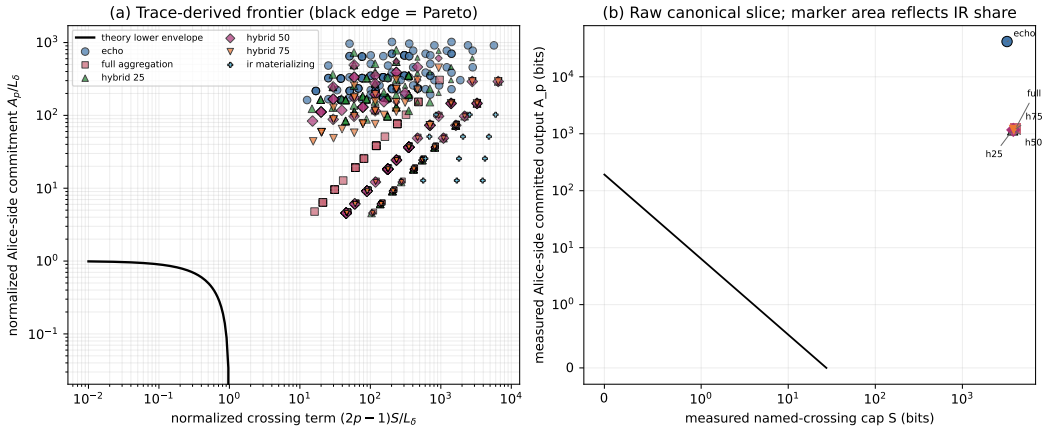



Figure S1: Complete event-traced W3 surface.

## S3 Matched representations and external semantic baselines

For each target the generator creates semantic aggregate, contiguous flat, round-robin flat, random commuting order, masked-share/update stream, locally folded flat, QASM bridge, TKET bridge, and ZX bridge forms. The representation record freezes its identifier, seed, generation parameters, serialized sizes, input certificate, and content hashes. Compilers see the same target basis, hardware abstraction, timeout, memory cap, and certificate.

The five W4 compilers are echo/pass-through, full aggregation, Qiskit optimization level 3, TKET PauliSimp, and PyZX full-reduce. W5 then applies eight frozen configurations: native and unified forms of the authors reference, Qiskit level-three local optimization, TKET PauliSimp, and PyZX full-reduce. Native-to-unified bridge deltas are measured before certificate comparison. A semantic-capable external output is grouped with the semantic side even when it is more compact than the authors' output.

The fifteen W4 predicate errors and thirty-three W5 predicate errors are retained in the canonical parquet files. They arise when an output lies outside the declared certificate predicate; they are neither correctness failures nor wins. No completed output bypasses the independent certificate.

## S4 Correctness certificate adversarial audit

The exact checker accepts commuting Pauli rotations in a tracked Clifford frame with rational-$\pi$ and declared formal-symbol coefficients. It moves a Pauli sign into the coefficient, reduces the

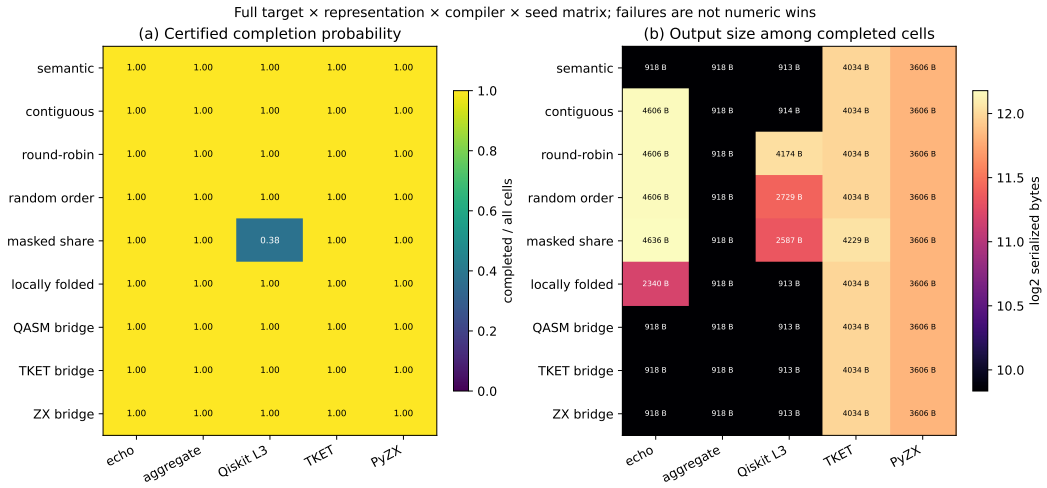



Figure S2: Full representation–compiler interaction summary; completion probability is separated from completed-cell metrics.

rational-$\pi$ component modulo $2\pi$, discards global phase, merges equal supports, and sorts addressed records deterministically. Equality of the canonical frame and rotation records implies projective unitary equality in the stated domain.

The mutation suite changes one angle, sign, support bit, qubit permutation, Clifford frame, deleted gate, or duplicated gate. Dense matrices through six qubits form an independent cross-check. The frozen result contains 56 mutations, 12 accepted and rejected cross-check cases, 0 false accepts, and no dense/symbolic disagreement. Floating-only parameters, unlisted gates, and noncommuting extracted networks are explicitly outside the acceptance domain.

## S5 Fixed-total-error QRE details

The budget schema requires

$$\epsilon_{\text{alg}} + \epsilon_{\text{synth}} + \epsilon_{\text{logical}} \leq \epsilon_{\text{total}}. \tag{S1}$$

The campaign uses the same fractional split in every pipeline and reallocates per-rotation synthesis accuracy after compilation. Equal-decimal allocation and a cost-optimized discrete allocation are both retained. The deterministic staq grid-synthesis executable runs with checking enabled; an angle cache is keyed by the exact angle/precision pair and records every use.

Table S3: QRE factorial controls.

| Dimension | Frozen levels |
|---|---|
| Total error | $10^{-3}, 10^{-6}, 10^{-9}$ |
| Allocation | equal decimal; cost optimized |
| QEC assumptions | conservative; improved surface-code profiles |
| Factory parallelism | serialized, low, medium, high Pareto points |
| Pipelines | semantic-first; materialize-first; external TKET PauliSimp |
| Outputs | rotations, $T$, qubits, factories, cycles, runtime, spacetime |

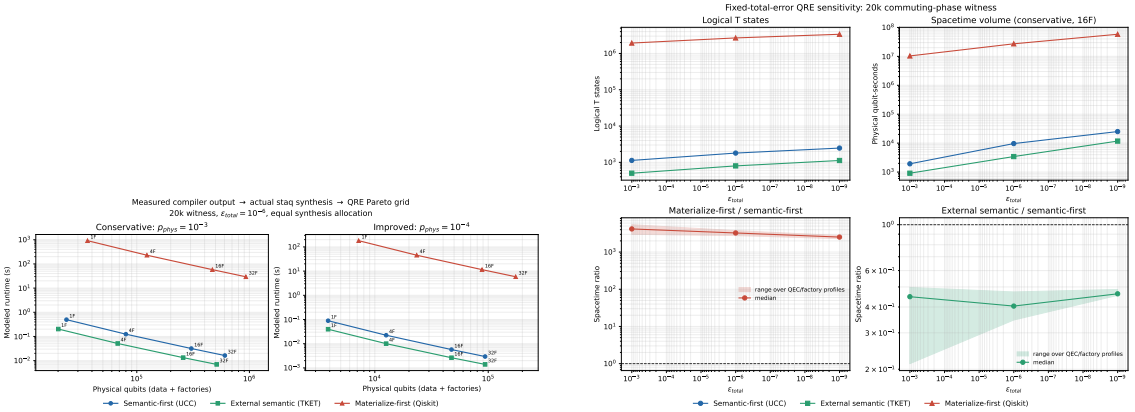



Figure S3: Pareto and sensitivity views generated from the complete W7 parquet file. Reversal or disappearance regions would remain visible; none is removed by filtering.

## S6 Natural workloads and complete ablation

The natural campaign is a full factorial over the declared families, scales, seeds, density levels, topologies, and all six binary mechanism toggles. Each cell records gates, depth, CX, runtime, RSS, input/output IR bytes, description bits, dense certificate, and the fixed-error QRE decomposition including data and factory physical qubits.

Table S4: Matched structured-family result retained without cherry-picking.

| Family | External-minus-authors gates [95% interval] | Strict success |
|---|---|---|
| controlled_power_ladder | 9.0 [5.7, 12.7] | yes |
| diagonal_hamiltonian | 38.0 [25.9, 51.8] | yes |
| phase_polynomial_blocks | -24.3 [-33.8, -16.3] | no |
| qaoa_ising | 44.7 [33.7, 56.6] | yes |
| qft_arithmetic | 8.1 [1.9, 13.8] | no |
| qpe_controlled_powers | -12.0 [-22.0, -3.0] | yes |
| trotter_commuting_blocks | 33.0 [17.6, 48.2] | yes |

Table S5: All six natural-family gate-count main effects. Off-minus-on is positive when enabling the factor reduces gates.

| Factor | Mean reduction | Bootstrap 95% interval |
|---|---|---|
| semantic_lift | 72.2 | [66.3, 78.1] |
| aggregation | 80.4 | [74.6, 86.4] |
| selector | 0.0 | [0.0, 0.0] |
| cache_reuse | 0.0 | [0.0, 0.0] |
| preset_recognizer | 364.9 | [349.8, 380.5] |
| projected_block_selection | 0.0 | [0.0, 0.0] |

The preset recognizer is the largest isolated gate-count contribution. Semantic lift and aggregation also have positive mean effects. Selector, cache/reuse, and projected block selection have zero isolated gate effect in this reference suite, and this null result is retained. Both negative controls have zero all-on-minus-all-off differences for gates, depth, CX, logical $T$, and spacetime, with no systematic-regression interval.

The line-topology preflight caught an invalid final-layout permutation in a candidate router, whose dense certificate failed before the frozen campaign. We replaced it with an explicit adjacent CX-SWAP construction restoring identity layout and reran the entire campaign. The failed preflight is retained because correctness takes precedence over favorable metrics.

## S7 Legacy and stress-result registry

The active submission body is the full-length `PaperDraft/main.tex`. The unintegrated 38-page source is frozen as `PaperDraft/legacy_main_v7_2.tex`, and the superseded nine-page reconstruction is frozen as `PaperDraft/short_main_w10.tex`. Neither frozen file is the submission source. The active manuscript retains useful earlier exact-semantics and stress diagnostics, but labels their provenance and does not silently mix them with the W3–W9 canonical headline tables.

Table S6: Disposition of legacy material.

| Material | Location | Submission disposition |
|---|---|---|
| Exact-semantics four-qubit witnesses | `appendix/appendix_C_prxq.tex` | Supplementary context only; not evidence for the $K$-ary asymptotic claim. |
| Finite-range rational CP period | `research/rational_witness/` | Retained certificate for the legacy exact-semantics corollary; not a W8 natural result. |
| Packing-width stress run | `research/packing_scaling/` | Historical theorem-linked stress evidence; superseded for headline measurements by W3/W4. |
| Width/topology external stress | `appendix/appendix_C_prxq.tex` | Status table retained; unsupported/predicate errors are not converted to quality. |
| Historical cut tracing | `research/cut_trace/` | Noncanonical after the event-schema revision; W3 event logs are canonical. |
| Earlier matched/natural tables | `research/matched_representation/`, `research/natural_factorial/` | Archived; W4 and W8 root-level parquet freezes are canonical. |
| Earlier per-rotation proxy QRE | `research/fixed_budget_qre/` | Excluded from the main claim; W7 fixed-total-error QRE supersedes it. |

This separation is deliberate. It preserves failed and superseded evidence without allowing duplicate filenames or stale values to support a new claim.

## S8 Data lineage and independent-directory reproduction

The normalized record key is

$$(\texttt{experiment_id}, \texttt{instance_id}, \texttt{representation_id}, \texttt{method_id}, \texttt{seed}, \texttt{backend_hash}, \texttt{tool_version}, \texttt{artifact_commit}, \texttt{config_hash}).$$

The index contains 24,748 unique records and retains 48 predicate-error rows. CSV/parquet mirror checks, duplicate-label detection, receipt source hashes, row counts, and orphaned frozen artifacts are checked by `scripts/audit_results.py`.

The clean-room runner copies a minimal artifact to an independent temporary directory, creates a new venv bound to the locked project packages, and works only from that directory. It reruns 9 headline-theorem tests, independently compiles and densely certifies one matched QAOA/Ising instance with both reference pipelines, and runs a fixed-error QRE subset that exercises 17 deterministic synthesis pairs. This is working- directory isolation, not a claim of binary independence from the host Python installation.

The committed repository is sufficient for this frozen-data reconstruction. The approximately 700-MB raw W3–W8 run tree is versioned separately in the private `OIerYangJZ/paper1-data` archive at the revision recorded in `data/manifest.yaml`; the audit verifies its registered hashes when that optional archive is materialized. The one-command rebuild does not claim to rerun the complete measurement campaigns.

Every W3–W9 headline experimental number in the main text and this supplement is a generated macro registered in `data/provenance_map.yaml`. Retained historical diagnostics keep

their original source paths and cannot support a new headline number unless registered. The complete machine-readable audit is `data/frozen/data_audit.json`; the clean-room receipt is `data/frozen/clean_room_receipt.json`.

## S9 Record codec and cut accounting

This section gives the field-level definitions behind the charged model of the main text. Every lower bound there holds for any self-delimiting serialization whose control field records the emitted-prefix length; the definitions below fix the serialization used by the upper bounds and by the instrumented implementation.

**Definition S9.1** (Sparse formal-parameter serialization)**.** *Fix formal angle symbols $\theta_1, \dots, \theta_M$. A parameter is stored in normalized sparse form*

$$\lambda = \sum_{\ell=1}^{s} c_\ell \theta_{j_\ell} + \frac{a}{d}\pi,$$
$$j_1 < \cdots < j_s, \quad c_\ell, a \in \mathbb{Z}, \quad d \geq 1,$$

*with zero coefficients omitted and* $\gcd(a, d) = 1$*. Let* $\mathsf{u}(z)$ *and* $\mathsf{s}(z)$ *be fixed self-delimiting codes for nonnegative and signed integers. The executable parameter serialization is*

$$\begin{aligned} \mathsf{ser}_{\mathrm{par}}(\lambda) = \mathsf{u}(s) \prod_{\ell=1}^{s} \big(\mathsf{u}(j_\ell)\mathsf{s}(c_\ell)\big) \\ \cdot \mathsf{s}(a)\mathsf{u}(d), \\ \mathsf{prec}(\lambda) := |\mathsf{ser}_{\mathrm{par}}(\lambda)|. \end{aligned} \tag{S2}$$

*Thus one nonzero formal coefficient costs $O(\log M)$ rather than $\Omega(M)$ bits. Qubit and generator locations are not part of this parameter payload; they are serialized by the complete-token codec below.*

**Definition S9.2** (Width-aware complete-token codec)**.** *Fix a self-delimiting integer code* $\mathsf{u}$*, a signed variant* $\mathsf{s}$*, a two-bit Pauli code* $\mathsf{p}$*, and a reserved record terminator* $\mathsf{d}$*. A token $\tau$ with gate-type ID $g$, arity $r$, ordered operand qubits $q_1, \dots, q_r$, sparse Pauli support $\{(z_\ell, \sigma_\ell)\}_{\ell=1}^{s}$, and $k$ parameter slots is encoded as*

$$\mathsf{ser}_{\mathrm{tok}}(\tau) = \mathsf{u}(g)\mathsf{u}(r) \prod_{i=1}^{r} \mathsf{u}(q_i)\ \mathsf{u}(s) \prod_{\ell=1}^{s} \big(\mathsf{u}(z_\ell)\mathsf{p}(\sigma_\ell)\big)$$
$$\cdot\, \mathsf{u}(k) \prod_{a=1}^{k} \big(\mathsf{u}(\mathrm{id}_a)\mathsf{ser}_{\mathrm{par}}(\lambda_a)\big)\mathsf{d}.$$

*The zero values of $r$, $s$, or $k$ are explicitly encoded, so the record is executable and uniquely parseable without out-of-band arity, support, or delimiter information. Update, IR-node, and output records use the same field order, with an opcode specifying which fields are active.*

*At growing width $n$, an arbitrary qubit or generator choice alone requires $\lceil \log_2 n \rceil$ bits in any injective codec; the concrete codec above uses $O(\log n)$ bits per listed address and $O(s \log n)$ bits for an $s$-sparse Pauli support. Consequently there is no width-independent "structural cost per token". The literal bit strings $\mathsf{ser}_{\mathrm{tok}}$ are charged wherever a token is live or emitted. Only when $n = n_0$ and a complete-token precision bound $q$ are both fixed is the token set $\Gamma_{B,n_0}^{\leq q}$ finite; the fixed-width exact-semantics statements abbreviate it as $\Gamma_B(q)$.*

**Definition S9.3** (Randomized dynamic-pass compiler and serialized cut budget)**.** *A compiler $\mathcal{A}$ may use random-access RAM, stacks, hash tables, caches, DAG/graph IRs, and intermediate tapes; it may also select a finite number $p = p(I, \rho)$ of scans or passes dynamically from the input $I$ and a finite random seed/tape $\rho$. The external input is append-only on its first exposure; a new forward pass is re-delivered by the input source from the start of the stream and is accounted for by the pass count (Definition 2). Any other access to a consumed prefix—a copy kept for a later scan through a*

*file, memory mapping, spool, or cache—is live compiler storage and is charged below. The final output is write-once. At every execution cut $c$, serialize a complete restart snapshot in five disjoint fields:*

- $\mathsf{ser}_{\mathrm{control}}(c)$*: format version, pass and finite-control IDs, dynamic pass count and termination/scheduling state, all input/output heads and emitted-prefix lengths, end flags, framing lengths, and the random seed/tape with its cursor;*
- $\mathsf{ser}_{\mathrm{store}}(c)$*: the persistent and ephemeral RAM, stacks, hash-table keys, bucket layout, cache contents and metadata, with parameter payloads and IR objects replaced by location tags;*
- $\mathsf{ser}_{\mathrm{window}}(c)$*: the ordered live read-window token structures, including the structural fields of Definition S9.2 but excluding parameter payloads;*
- $\mathsf{ser}_{\mathrm{parameter}}(c)$*: the location-tagged concatenation of every live parameter payload in control, store, window, and IR, each encoded by* (S2)*;*
- $\mathsf{ser}_{\mathrm{IR}}(c)$*: every other live intermediate tape/DAG/table object and every rereadable consumed-input block that can affect future execution, serialized by node opcode, the structural fields of Definition S9.2, edge endpoints, multiplicity, and object boundaries, but with parameter payloads omitted.*

*The corresponding bit-level budget is*

$$\boxed{\begin{aligned} B_{\mathrm{cut}}(c) = {} & B_{\mathrm{control}}(c) + B_{\mathrm{store}}(c) \\ & + B_{\mathrm{window}}(c) + B_{\mathrm{parameter}}(c) + B_{\mathrm{IR}}(c). \end{aligned}} \tag{S3}$$

*where each summand is the literal bit length of the named serialization. Parameter payloads are the formal angle parameters of live tokens and IR objects; raw integer data that a compiler computes and keeps, such as residues or partial sums held in a packed RAM word, belong to* $\mathsf{ser}_{\mathrm{store}}$ *and are charged at their literal bit length. Every upper bound below uses this convention, and no lower bound depends on it, because the lower bounds apply to any self-delimiting serialization whose control field records the emitted-prefix length. For a fixed run,* $B_{\mathrm{cut}}$ *is the maximum over its cuts. For randomized compilers we use the worst-input expected budget*

$$\overline{B}_{\mathrm{cut}}(\mathcal{A}, \mathcal{I}) := \max_{I \in \mathcal{I}} \mathbb{E}_{\rho}\big[\max_{c} B_{\mathrm{cut}}(c; I, \rho)\big]. \tag{S4}$$

*No theorem below substitutes a single field, such as* $\max_c B_{\mathrm{parameter}}(c)$ *or* $\max_c B_{\mathrm{IR}}(c)$*, for the total* $B_{\mathrm{cut}}$*. The final output is write-once, is a codeword of a fixed prefix-free output code, and is charged separately by the output codes below.*

**Remark S9.4** (Model scope and escape channels)**.** *Equation* (S3) *charges all restart information, not only the finite-control state. In particular, a ZX graph, phase-polynomial table, spilled update log, hash table, cache, random seed, or materialized intermediate tape is serialized in the indicated field; formal angle parameters are serialized in* $B_{\mathrm{parameter}}$ *and computed integer data in* $B_{\mathrm{store}}$*, so every coefficient is charged exactly once. Internal rescanning of retained data is allowed, but the pass schedule, heads, and any retained copy of old input are part of the snapshot. A pass re-delivered by the source is accounted by the pass count rather than by the snapshot, so a bound read off a single cut, such as Theorem 3, requires the stream to be delivered once; the transcript bounds of Section III cover re-delivered passes. The five fields are disjoint by construction, so parameter bits are not counted again as store, window, or IR structure. A compiler may escape a RAM-only workspace lower bound by materializing a large IR, but it then pays the same bits in* $B_{\mathrm{cut}}$*.*

**Remark S9.5** (Physical scope of the bounded-memory streaming model)**.** *The charged streaming model of Definition S9.3—bounded live state, explicit pass accounting, append-only first exposure—is the operating regime of a real-time fault-tolerant control stack, not a convenience abstraction. Between rounds, the classical electronics adjacent to the quantum device holds decoder state, Pauli-frame records, and instruction buffers under hard latency and buffer budgets: Pauli corrections can be tracked in a classical frame instead of being applied, which relaxes the timing constraints*

| Formal object | Codec object/file | Included information | Measured field |
|---|---|---|---|
| semantic input | `SemanticStream` | width and address-complete aggregate records | input artifact bits |
| dispersed flat input | `FlatUpdateStream` | width, round count, and every additive update | input artifact bits |
| $\mathsf{ser}_{\mathrm{control}}$ | `ControlState/control.uccbin` | pass count/index, heads, scheduling, status, framing and seed state when present | $B_{\mathrm{control}}$ |
| $\mathsf{ser}_{\mathrm{store}}$ | `StoreState/store.uccbin` | aggregate tables, maps, caches, stacks and their layout, excluding tagged parameter/IR payloads | $B_{\mathrm{store}}$ |
| $\mathsf{ser}_{\mathrm{window}}$ | `LiveWindow/window.uccbin` | ordered live complete-token structure and explicit parameter-slot count | $B_{\mathrm{window}}$ |
| $\mathsf{ser}_{\mathrm{parameter}}$ | `ParameterLedger/parameter.uccbin` | sparse symbolic parameters under complete location tuples | $B_{\mathrm{parameter}}$ |
| $\mathsf{ser}_{\mathrm{IR}}$ | `IRField/ir.uccbin` | serialized DAG width, opcodes, operands, nodes, edges, ports, boundaries and metadata, with parameters removed | $B_{\mathrm{IR}}$ |
| committed prefix | `committed-prefix.bin` | literal immutable prefix of a codec output frame | $B_{\mathrm{com}}$ |
| external output | `ExternalToolOutput` | tool, format and literal API payload | output artifact bits |
| certificate input | `CertificateInput` | certificate kind, certified-artifact SHA-256 and literal sidecar | certificate artifact bits |
| pass count | manifest `passes.count` and row `p` | actual declared scans of the instrumented compiler | $p$ |

Table S7: One-to-one map from formal serialized objects to executable artifacts. No row permits a gate, token, support, or IR-node count to stand in for bits.

*on syndrome measurement and decoding [1], but they must be resolved before the next non-Clifford gate [2], and surface-code compilers are built as streaming pipelines that target real-time operation [3]. Distributed and modular architectures motivate the same scope from the interconnect side: a cross-module interaction splits the* support *of a gate across modules while its coefficient stays within one module, so modularity bounds the state that crosses a partition without by itself dispersing any coefficient. These architectures therefore ground the model's memory and pass constraints only; no claim is made that modular partitioning instantiates the dispersed streams of Section IV, whose instances come from scheduling and twirling alone.*

## S9.1 Executable Model–Implementation Correspondence

The formal fields above are implemented by one canonical binary codec rather than by dimension-changing estimates. The executable specification is `encoding/FORMAT.md`; its reference implementation is `encoding/codec.py`, and the only bit-accounting entry point is `instrumentation/resource_accounting.py`. A complete codec frame has the form

$$\texttt{UCCACCTO} \,\|\, \mathsf{u}(\text{version}) \,\|\, \mathsf{u}(\text{type}) \,\|\, \mathsf{u}(|z|) \,\|\, z, \tag{S5}$$

where the displayed terminal zero denotes a null byte, $\mathsf{u}$ is the canonical base-128 unsigned varint, and $z$ is a typed canonical payload. Signed integers use zigzag coding; rational coefficients are reduced signed numerator/positive-denominator pairs. Maps are ordered by their UTF-8 key bytes. Qubit, generator, symbol, location, node, edge-endpoint, port, pass, and head addresses are literal varints in $z$.

**Lemma S9.6** (Canonical prefix-free implementation)**.** *Version-one complete frames are injective and prefix-free. Concatenations of frames are uniquely parseable without an out-of-band arity or record count. The semantic and flat decoders preserve their exact aggregate coefficient table.*

*Proof.* Canonical varints have a unique terminating byte, and the version, type, and payload length are therefore uniquely decoded. The declared payload length locates the end of the frame. A complete valid frame cannot be a proper prefix of another complete valid frame: the shorter declared payload ends at its frame boundary, whereas any following byte belongs to a subsequent frame. Typed payload values have distinct one-byte tags; lists and maps carry counts, maps have

strictly increasing keys, and reduced rational pairs are unique. This proves injectivity and unique concatenated parsing. Finally, `semantic_signature` sums exact rational sparse parameters by the literal pair (generator address, Pauli support), so encode–decode leaves the aggregate table unchanged for both input representations. □

**Proposition S9.7** (File-backed accounting soundness)**.** *Suppose the version-one verifier accepts a cut manifest $M$. For every $f \in \{\text{control}, \text{store}, \text{window}, \text{parameter}, \text{IR}, \text{com}\}$, let $A_f$ be the file named by $M$. Then the implementation reports*

$$\begin{aligned} B_f &= 8|A_f|_{\text{byte}}, \\ B_{\text{cut}} &= B_{\text{control}} + B_{\text{store}} + B_{\text{window}} \\ &\quad + B_{\text{parameter}} + B_{\text{IR}}, \end{aligned} \tag{S6}$$

*and reports the manifest pass count as $p$. The reported values change when a serialized address crosses a varint-width boundary. They cannot be obtained from a node or gate count alone.*

*Proof.* The accountant first writes the five complete frames and the literal committed prefix, then obtains every byte count by `stat`, stores the SHA-256 of each file, writes $M$, and invokes the verifier before returning a row. The verifier reopens all six files, recomputes sizes and digests, decodes the five restart fields, and rejects any disagreement. It then forms the sum in (S6); there is no count-to-bit code path. Since an address is a literal canonical varint, for example 127 and 128 occupy different address lengths. Conversely, two one-node DAGs may contain those different addresses and hence have unequal file lengths despite equal node count. □

**Remark S9.8** (Observable boundary versus restart cut)**.** *The masked-share tracer serializes the five declared cut fields after every update. The verifier reopens and decodes every recorded field slice, checks its size and digest, and recomputes the cut sum; its maximum is therefore a file-backed measurement of the declared $B_{\text{cut}}$ coordinate. The W3 campaign does not claim that the Python process is destroyed and restarted between events. External compilers do not expose their private in-process DAGs, caches, or tapes. Their matched-matrix and natural-factorial rows consequently report a file-backed* API-boundary *snapshot and are diagnostic only; they are not promoted to measurements of the maximum private $B_{\text{cut}}$. Peak RSS remains a separate byte dimension and is not substituted for any field in* (S3)*.*

**Remark S9.9** (Self-contained and conditional descriptions)**.** *Both input-stream objects declare either `self_contained` or `conditional`. The former forbids an external dictionary. The latter requires the SHA-256 of the exact public family dictionary, and its manifest records the dictionary's separate file size and whether it is charged at the cut. Thus a conditional description cannot silently be reported as a self-contained circuit code.*

## S9.2 Restart from the serialized cut

**Lemma S9.10** (Restart from the serialized cut)**.** *Fix $\mathcal{A}$ as in Definition S9.3, receiving a stream of Definition 1 once. At the crossing $c_{A\to B}$ before the last round, the five fields in* (S3)*, together with any prefix already committed to the final write-once output, determine the continuation of that run for every supplied last round $v$. For a randomized run this statement is conditional on the serialized seed/tape and cursor.*

*Proof.* The control field restores the exact transition, all head positions, dynamic pass/termination schedule, and random seed/tape cursor. The store and window fields restore RAM, stack, hash/cache metadata, nonparameter machine data, and ordered live token structures. The parameter field restores every omitted coefficient by its location tag. The IR field restores every other live intermediate tape, DAG, table, addressed record, and any consumed input block retained for rescanning. These disjoint fields therefore reconstruct the complete machine configuration. Conditional on the serialized random tape, the receiver can append the supplied second round and execute the same dynamically chosen continuation. The separately transmitted final-output prefix is prepended to the suffix produced after restart. No additional state, old-input file, or public-family side channel is used. The single-delivery hypothesis is needed: if the source re-delivered the stream for a later pass, the continuation would reread the first round, which the receiver does not hold. □

## S10 Certificate domain and soundness

Let

$$\mathcal{C}_n = \langle H, S, S^\dagger, X, Y, Z, \mathrm{CX}, \mathrm{CZ}, \mathrm{SWAP} \rangle$$

be the Clifford gate language on addressed qubits. Fix a declared finite formal-symbol set $\mathcal{S}$ and the exact coefficient module

$$\Lambda = \mathbb{Q}\pi \oplus \bigoplus_{s \in \mathcal{S}} \mathbb{Q}\theta_s.$$

For a signed Hermitian $n$-qubit Pauli $P$ and $\lambda \in \Lambda$, write $R_P(\lambda) = \exp(-i\lambda P/2)$.

**Definition S10.1** (Exact commuting-Pauli/Clifford certificate domain)**.** *The domain $\mathcal{D}_{\mathrm{CPC}}$ consists of finite addressed circuits over $\mathcal{C}_n$, exact records $R_Z(\lambda)$ and $R_P(\lambda)$, identity and barrier records, and an arbitrary exact global phase. Every coefficient is a reduced rational $\pi$ component plus a sorted sparse list of reduced rational formal-symbol components carried by the input; a floating-only parameter is not in the exact domain.*

*Scan a circuit in execution order while maintaining a Clifford frame $F$ and the invariant*

$$U_{\mathrm{prefix}} = F R_{P_t}(\lambda_t) \cdots R_{P_1}(\lambda_1) \quad \text{up to global phase.} \tag{S7}$$

*The frame is represented by the complete signed inverse-conjugation tableau $\{F^\dagger X_j F, F^\dagger Z_j F\}_{j=1}^n$. Appending a physical rotation $R_Q(\lambda)$ appends $F^\dagger Q F$ to the Pauli list. The certificate predicate additionally requires all extracted $P_j$ to commute pairwise.*

*Canonicalization moves a Pauli sign into its coefficient, discards an identity-axis rotation, adds coefficients on identical unsigned supports, reduces the rational $\pi$ coefficient modulo 2 to the interval $[0, 2)$, keeps every formal-symbol coefficient exact, deletes zero records, and sorts the remaining records lexicographically by their binary $(x, z)$ masks. Circuit and Clifford global phases are omitted. The resulting record is*

$$\begin{aligned} \beta_j &:= (b_{j,s})_{s \in \mathcal{S}} \in \mathbb{Q}^{\mathcal{S}}, & \bar{a}_j &\in \mathbb{Q} \cap [0, 2), \\ \mathrm{can}(C) &= \big(n, T_F, ((x_j, z_j, \bar{a}_j, \beta_j))_{j=1}^{\ell}\big). \end{aligned} \tag{S8}$$

*The symbolic checker accepts exactly when the two canonical byte records are equal.*

**Proposition S10.2** (Soundness of canonical symbolic acceptance)**.** *For $C, C' \in \mathcal{D}_{\mathrm{CPC}}$ satisfying the pairwise-commutation predicate,*

$$\mathrm{can}(C) = \mathrm{can}(C') \quad \Longrightarrow \quad U_C = e^{i\phi} U_{C'}$$

*for some $\phi \in \mathbb{R}$.*

*Proof.* Induction over the input records proves (S7). For a Clifford record $G$, replace $F$ by $GF$. For a rotation,

$$R_Q(\lambda) F = F R_{F^\dagger Q F}(\lambda),$$

which is exactly the tableau update used by the checker. Thus the extracted record implements the original circuit up to its explicitly ignored global phase.

Pairwise commutation permits deterministic reordering without changing the unitary. Sign normalization is exact because $R_{-P}(\lambda) = R_P(-\lambda)$, while coefficient reduction changes a factor only by

$$R_P(\lambda + 2k\pi) = (-1)^k R_P(\lambda).$$

All such signs multiply to a global phase; an identity-axis rotation is also a global phase. Equality of the sorted rotation records, including every formal-symbol component, therefore gives equal rotation products projectively.

Finally, equality of the signed tableaus means that $F' F^\dagger$ commutes with every Pauli generator. The Pauli representation generates the full matrix algebra, so $F' F^\dagger$ is a scalar. Hence the two Clifford frames also agree up to global phase, proving the claim. □

**Remark S10.3** (Status and completeness boundary)**.** *Only status* `completed_valid`*, accompanied by the content-addressed canonical records and an independent certificate payload, is an acceptance. A canonical mismatch is* `completed_invalid`*; an unlisted gate or malformed address is* `unsupported`*; failure of pairwise commutation is* `predicate_error`*; and a floating coefficient without an exact sidecar, an unassigned formal symbol in the dense checker, or a dense distance inside the numerical guard band is* `numerical_inconclusive`*. Only* `completed_valid` *rows enter quality means. The proposition is a soundness statement, not a claim that every equivalent Clifford-plus-rotation circuit has the same record.*

For $n \le 6$, the audit independently forms dense matrices and checks the normalized Frobenius projective distance

$$d_{\mathrm{F,proj}}(U,V) = 2^{-n/2} \min_{|\omega|=1} \|U - \omega V\|_{\mathrm{F}}.$$

The frozen suite covers both accepted and rejected pairs at every width $1 \le n \le 6$ and changes, separately, an angle, a sign, one support bit, a qubit permutation, the Clifford frame, and a deleted or duplicated gate. The executable definition and status contract are `certificates/CERTIFICATE_SPEC.md`; the canonicalizer and checker are `certificates/canonicalize.py` and `certificates/symbolic_checker.py`.

## S11 Exact-semantics results for fixed-width circuits

The main text concerns growing width and accuracy $\epsilon > 0$. For completeness we record the exact-semantics ($\epsilon = 0$), fixed-width results for finite-state transducers, with full proofs.

**Remark S11.1** (Separate fixed-width and growing-width encoding regimes)**.** *The exact corollary in this subsection fixes $n = n_0$, a precision bound $q$, a deterministic constant pass count $p_0$, and one finite alphabet $\Gamma^{\le q}_{B,n_0}$; its unit is therefore a token over that fixed alphabet. A* finite-window, finite-burst transducer *is, in this regime, a deterministic machine whose pass $i$ reads the tape written by pass $i-1$ (pass 1 reads the lowered input) and writes a new tape; each pass is a subsequential transducer over $\Gamma^{\le q}_{B,n_0}$ whose state consists of finite control, a persistent store, and a window of at most $w$ tokens, emits at most $\mu$ tokens per step, and has transitions that do not depend on head positions.* Constant budget *means that $p_0$, $w$, $\mu$, and the set of reachable states are independent of the repetition count $r$; head positions and intermediate tapes are not part of this budget. No such constant-cost token assumption is made in the growing-width results. There $n = m + 1$ (or $n \ge m$), the pass count may be $p = p(m, I, \rho)$, and every input, output, RAM, IR record, and head is charged in bits by Definition S9.2 and* (S3)*.*

**Definition S11.2** (Fixed-width witness family)**.** *Let $H_\Lambda$ be a fixed invertible boundary layer (a Hadamard layer in the experiments) and let $D = \prod_{j=1}^{m_0} \exp(-i\theta_j P_j/2)$ be a fixed commuting diagonal layer with Pauli-Z characters $P_j = Z^{a_j}$, $a_j \in \mathbb{F}_2^n$, and angles $\theta_j$. The witness at repetition count $r$ is $C_r = H_\Lambda D^r H_\Lambda^{-1}$, with exact lowering $\mathcal{L}_B(C_r) = \Pi M^r \Sigma$ for fixed token strings $\Pi, M, \Sigma$.*

Two instances are certified and reused here. The *rational CP witness* ($n = 4$; ten $rz/cp$ terms; $\mathrm{rank}_{\mathbb{F}_2}(A) = 4$) satisfies finite-range noncollision with exact period $T = 131040$, certified by the self-contained rational angle, single-term order, relative-phase, and LCM derivation in Section S11.1. The *irrational independent-Pauli witness* ($n = 4$; supports $1100, 0110, 0011, 1000$; $\theta_1/\pi = \sqrt{2}/11$) satisfies unbounded noncollision by the following uniqueness lemma.

**Lemma S11.3** (Coefficient uniqueness for independent characters)**.** *If $a_1, \dots, a_{m_0} \in \mathbb{F}_2^n$ are linearly independent and two diagonal layers with coefficient vectors $\alpha, \alpha'$ satisfy $D(\alpha) = e^{i\varphi} D(\alpha')$, then $\alpha_j \equiv \alpha'_j \pmod{2\pi}$ for every $j$. In particular, if some $\theta_j/\pi$ is irrational, the powers $D^r$ are pairwise distinct up to global phase for all $r \ge 1$.*

*Proof.* Independence makes $z \mapsto (\chi_{a_1}(z), \dots, \chi_{a_{m_0}}(z))$ surjective onto $\{\pm 1\}^{m_0}$. Comparing the phase constraint at two sign patterns that differ only in coordinate $j$ gives $2(\alpha_j - \alpha'_j) \equiv 0 \pmod{4\pi}$. For the second claim, $(r - r')\theta_j \in 2\pi\mathbb{Z}$ with $\theta_j/\pi$ irrational forces $r = r'$. □

**Assumption S11.4** (Finite-range noncollision)**.** *On a range $1 \le r \le R$, the powers $D^1, \dots, D^R$ are pairwise distinct up to global phase.*

**Assumption S11.5** (Unbounded noncollision)**.** *The powers $D^r$, $r \geq 1$, are pairwise distinct up to global phase.*

**Corollary S11.6** (Exact-semantics final-output lower bound)**.** *Assume exact lowering ($\epsilon = 0$), a fixed-width witness $C_r = H_\Lambda D^r H_\Lambda^{-1}$ with $\mathcal{L}_B(C_r) = \Pi M^r \Sigma$, and unbounded noncollision (Definition S11.5). Let $\mathcal{A}$ be a uniform deterministic finite-window, finite-burst transducer with constant budget in the sense of Definition S11.1 (constant $p, w, \mu$ and a finite reachable state set; head positions are not read) that is correct for every $r \geq 1$. Then there exist constants $h \geq 1$, $D \geq 1$ (depending on $\mathcal{A}$ and the witness but not $r$) such that for every $r \geq h + D$,*

$$|T_p(r)| \geq \frac{r - h - D + 1}{D} = \Omega_{\mathcal{A}}(r). \tag{S9}$$

*Proof.* This is Definition S11.16, proved in full in Section S11.2. Constant budget gives one finite charged alphabet; per-tape pumpability propagation gives pumpability of the final output; noncollision prevents collapse of distinct pump residues. □

**Corollary S11.7** (Finite-range tail form)**.** *Let $\mathcal{A}$ be as in Definition S11.6 but correct only for $1 \leq r \leq R$, under finite-range noncollision (Definition S11.4) with $N(R) = R$. Then there are constants $h, D$, given by the pumpability data of $\mathcal{A}$'s final pair family, with $|T_p(r)| \geq (r-h-D+1)/D$ for $h + D \leq r \leq R$ (Definition S11.17). For the rational CP witness, whose exact period is $T = 131040$, this applies for $R \leq T$.*

**Corollary S11.8** (Counting toll for short outputs)**.** *If finite-range noncollision supplies $N(R)$ distinct targets, then every deterministic exact compiler satisfies (Definition S11.18)*

$$\max_{1 \leq r \leq R} B_{\text{self}}(T_p(r)) \geq \lceil \log_2 N(R) \rceil .$$

*The same statement holds for any single fixed prefix-free output code, in particular for a family-relative code with one public dictionary used on the entire range.*

The irrational independent-Pauli witness instantiates Definition S11.6 for all $r$ via Definition S11.3; the rational CP witness instantiates Definition S11.7 on ranges $R \leq T$. These exact-semantics results concern $\epsilon = 0$ and fixed width $n = 4$; the results below generalize to growing $m$ and expose the dependence on $\epsilon > 0$.

## S11.1 Self-Contained Rational CP Period Certificate

The canonical four-qubit rational witness has diagonal block

$$\begin{aligned} D = \prod_{j=0}^{3} RZ_j(\theta_j) \prod_{0 \leq a < b \leq 3} CP_{ab}(\theta_{ab}), \\ \boldsymbol{\theta}_{RZ}/\pi = (1/7, 1/8, 1/9, 1/10), \\ (\theta_{01}, \theta_{02}, \theta_{03})/\pi = (1/12, 1/13, 1/14), \\ (\theta_{12}, \theta_{13}, \theta_{23})/\pi = (1/14, 1/15, 1/16). \end{aligned}$$

For a reduced rational angle $\theta/\pi = p/q$, both $RZ(\theta)$ and $CP(\theta)$ have projective single-term order

$$\text{ord}(p/q) = \frac{2q}{\gcd(|p|, 2q)},$$

because the relevant relative phase is $p/q$ modulo 2. The complete single-term certificate is

| term | support | $\theta/\pi$ | $T_j$ |
|---|---|---|---|
| $RZ_0$ | 0 | 1/7 | 14 |
| $RZ_1$ | 1 | 1/8 | 16 |
| $RZ_2$ | 2 | 1/9 | 18 |
| $RZ_3$ | 3 | 1/10 | 20 |
| $CP_{01}$ | 01 | 1/12 | 24 |
| $CP_{02}$ | 02 | 1/13 | 26 |
| $CP_{03}$ | 03 | 1/14 | 28 |
| $CP_{12}$ | 12 | 1/14 | 28 |
| $CP_{13}$ | 13 | 1/15 | 30 |
| $CP_{23}$ | 23 | 1/16 | 32 |

Hence

$$\begin{aligned} T_{\text{terms}} &= \operatorname{lcm}(14, 16, 18, 20, 24, 26, 28, 28, 30, 32) \\ &= 2^5 \cdot 3^2 \cdot 5 \cdot 7 \cdot 13 = 131040, \end{aligned}$$

which proves that $D^{131040}$ is a global phase. To certify minimality for the product rather than merely obtain this common upper bound, subtract the phase of 0000. For $z = z_0 z_1 z_2 z_3$, the exact relative phase is

$$\begin{aligned} w(z) = {} & z_0/7 + z_1/8 + z_2/9 + z_3/10 \\ & + z_0 z_1/12 + z_0 z_2/13 + z_0 z_3/14 \\ & + z_1 z_2/14 + z_1 z_3/15 + z_2 z_3/16 \pmod 2. \end{aligned}$$

In particular,

$$\begin{aligned} w(1011) &= \frac{37007}{65520}, \\ \operatorname{ord}(w(1011)) &= \frac{2 \cdot 65520}{\gcd(37007, 2 \cdot 65520)} \\ &= 131040. \end{aligned}$$

Thus no smaller positive power of $D$ is a global phase, and $T = 131040$ exactly. It follows that $D^1, \ldots, D^R$ are pairwise distinct up to global phase for every $R \le T$, since $D^i \simeq D^j$ requires $T \mid j - i$.

The machine certificate uses only Python `fractions.Fraction` and prints all 16 relative numerators, denominators, and orders. The frozen integer output contains

`term_order_lcm=131040`,
`relative_order_lcm=131040`,
`minimality_state_1011_order=131040`,
`exact_global_phase_period_T=131040`.

The reproducibility files and hashes are listed below; each digest is split at its 32-hex midpoint only for typesetting.

- verifier: `research/rational_witness/verify_period.py`,
  SHA256 `c7c90f26e00e515383e94e4bbd707574eb3aabf94ad71c3cd5e2cdb8d5db7e76`;
- integer output: `research/rational_witness/period_certificate.txt`,
  SHA256 `375295d776f21ea2a82ee4f2bab5205d25bfef85ee864900baacfbd80be9bd6e`;
- manifest: `research/rational_witness/SHA256SUMS`.

From the repaired-manuscript root, the byte-for-byte check is

```
cd research/rational_witness
python3 verify_period.py \
-check period_certificate.txt
```

## S11.2 Full Proofs for the Transducer Lower Bound

This section supplies the full finite-state proof layer underlying Definitions S11.6 to S11.8. The main text states the shorter version; the appendix keeps the pumpability data explicit. The following statements restate the main-text theorem and corollaries with full hypotheses and complete proofs; they are not additional independent results.

### S11.2.1 Pumpable Pair Families

Persistent store can depend on $r$, so the induction tracks both a store value and a tape word. In this subsection, $D$ in pumpability data denotes a period modulus, not the diagonal witness layer in $C_r = H_\Lambda D^r H_\Lambda^{-1}$. The transducer model is the one of Definition S11.1. The next definition records pumpability with explicit residue data.

**Definition S11.9** (Pumpable pair family)**.** *Let $\Gamma$ be a finite alphabet and $P$ a finite set of store values. A family*

$$X(r) = (\beta(r), W(r)) \in P \times \Gamma^*, \qquad r \in \mathbb{N},$$

*is* pumpable *if there exist integers $h \geq 0$ and $D \geq 1$ such that for every residue $\rho \in \{0, \dots, D-1\}$ there are*

$$\beta_\rho \in P, \qquad A_\rho, B_\rho, C_\rho \in \Gamma^*,$$

*with*

$$X(h + \rho + kD) = (\beta_\rho, A_\rho B_\rho^k C_\rho)$$

*for every $k \geq 0$. The word $B_\rho$ is the pump word on residue $\rho$. The residue is* live *if $B_\rho \neq \epsilon$ and* collapsed *if $B_\rho = \epsilon$.*

**Lemma S11.10** (Input pumpability)**.** *For the fixed-width witness $T_0(r) = \Pi M^r \Sigma$, the pair family consisting of the initial persistent store and $T_0(r)$ is pumpable with $h = 0$, $D = 1$, pump word $M$, and fixed store value.*

*Proof.* Take $A_0 = \Pi$, $B_0 = M$, and $C_0 = \Sigma$. □

### S11.2.2 One-Pass Propagation

Fix a precision bound $q$ and a charged alphabet $\Gamma = \Gamma_B(q)$. Under this bound, a pass of $\mathcal{A}$ is an ordinary deterministic subsequential transducer over a finite alphabet whose transitions do not read head positions (Definition S11.1). Its finite state consists of the ephemeral state, the persistent store, and the $w$-token window. The end-of-pass persistent store is one component of the final finite state.

**Lemma S11.11** (One-pass pumpability propagation)**.** *Let $X(r) = (\beta(r), W(r))$ be a pumpable pair family over a finite input alphabet. Let $\tau$ be one deterministic finite-window, finite-burst pass whose output alphabet and store set are finite. Let*

$$\tau(\beta(r), W(r)) = (\beta'(r), W'(r))$$

*be the resulting final store and output tape. Then $X'(r) = (\beta'(r), W'(r))$ is pumpable.*

*Proof.* Fix pumpability data $h, D, \{\beta_\rho, A_\rho, B_\rho, C_\rho\}_\rho$ for $X$. We first treat one residue $\rho$, then recombine all residues into $r$-indexed pumpability data for $X'$.

For a fixed residue $\rho$, write

$$W_k = AB^kC, \qquad \beta_k = \beta,$$

where $k \geq 0$ and $A, B, C, \beta$ abbreviate the residue data. Absorb the ephemeral state, persistent store, and window into the pass's finite state set $S$. Starting from the state determined by $\beta$, process $A$. This emits a fixed word $O_A$ and reaches a state $s_A \in S$.

Let $f : S \to S$ be the state map obtained by reading the block $B$, and let $o(s) \in (\Gamma')^*$ be the output emitted while reading $B$ starting in state $s$. The sequence

$$s_A,\ f(s_A),\ f^2(s_A), \dots$$

in the finite set $S$ is ultimately periodic. Thus there exist $u \geq 0$ and $\ell \geq 1$ with $u + \ell \leq |S| + 1$ such that

$$f^{t+\ell}(s_A) = f^t(s_A) \qquad \text{for all } t \geq u.$$

For $k = u + v\ell + \eta$ with $0 \leq \eta < \ell$, the output while reading $B^k$ is

$$\begin{aligned}&\big(o(s_A)o(f(s_A))\cdots o(f^{u-1}(s_A))\big)\\ &\quad\cdot\Big(o(f^u(s_A))\cdots o(f^{u+\ell-1}(s_A))\Big)^v\\ &\quad\cdot\big(o(f^u(s_A))\cdots o(f^{u+\eta-1}(s_A))\big).\end{aligned}$$

After this, the pass reads the fixed suffix $C$, emits a word depending only on the state $f^{u+\eta}(s_A)$, and ends in a persistent store value depending only on the same state.

Restore the residue subscripts. For each original residue $\rho \in \{0, \ldots, D-1\}$, let $u_\rho \geq 0$ and $\ell_\rho \geq 1$ be the preperiod and cycle length obtained above, let $f_\rho$, $s_{A_\rho}$, and $O_{A_\rho}$ be the corresponding state map, post-prefix state, and prefix output. For $0 \leq \eta < \ell_\rho$, let $F_{\rho,\eta}$ be the complete word emitted while reading the fixed suffix $C_\rho$ from state $f_\rho^{u_\rho+\eta}(s_{A_\rho})$, including the end-of-tape finalizer output, and define

$$\begin{aligned}P_\rho &:= o\big(s_{A_\rho}\big)o\big(f_\rho(s_{A_\rho})\big)\cdots o\big(f_\rho^{u_\rho-1}(s_{A_\rho})\big),\\ Q_\rho &:= o\big(f_\rho^{u_\rho}(s_{A_\rho})\big)\cdots o\big(f_\rho^{u_\rho+\ell_\rho-1}(s_{A_\rho})\big),\\ R_{\rho,\eta} &:= o\big(f_\rho^{u_\rho}(s_{A_\rho})\big)\cdots o\big(f_\rho^{u_\rho+\eta-1}(s_{A_\rho})\big)\cdot F_{\rho,\eta}.\end{aligned}$$

Let $\beta'_{\rho,\eta}$ be the end-of-pass persistent store value reached from that state. Define

$$\begin{aligned}u^* &:= \max_\rho u_\rho, \qquad \ell^* := \operatorname{lcm}_\rho \ell_\rho,\\ h' &:= h + Du^*, \quad D' := D\ell^*.\end{aligned}$$

Each new residue $\rho' \in \{0, \ldots, D'-1\}$ decomposes uniquely as

$$\rho' = \rho + D\tilde{\eta}, \qquad \rho \in \{0, \ldots, D-1\}, \quad \tilde{\eta} \in \{0, \ldots, \ell^*-1\}.$$

For $r = h' + \rho' + k'D'$ with $k' \geq 0$, the old representation $r = h + \rho + kD$ has

$$k = u^* + \tilde{\eta} + k'\ell^*.$$

Since $u^* \geq u_\rho$, we have $k \geq u_\rho$ for every $k' \geq 0$; and since $\ell_\rho \mid \ell^*$, the residue of $k - u_\rho$ modulo $\ell_\rho$ does not depend on $k'$. Set

$$\eta := (u^* + \tilde{\eta} - u_\rho) \bmod \ell_\rho, \qquad v_0 := \Big\lfloor \frac{u^* + \tilde{\eta} - u_\rho}{\ell_\rho} \Big\rfloor \geq 0,$$

so that

$$k = u_\rho + (v_0 + k'\ell^*/\ell_\rho)\ell_\rho + \eta.$$

By the output decomposition above, for every $k' \geq 0$,

$$X'(h' + \rho' + k'D') = \big(\beta'_{\rho,\eta},\ A'_{\rho'}(B'_{\rho'})^{k'}C'_{\rho'}\big),$$

where

$$A'_{\rho'} := O_{A_\rho}P_\rho Q_\rho^{v_0}, \qquad B'_{\rho'} := Q_\rho^{\ell^*/\ell_\rho}, \qquad C'_{\rho'} := R_{\rho,\eta}.$$

This is exactly pumpability of $X'$ with data $(h', D')$ and residue data $\{\beta'_{\rho,\eta}, A'_{\rho'}, B'_{\rho'}, C'_{\rho'}\}_{\rho'}$. □

**Corollary S11.12** (Per-tape induction, for proof)**.** *Assume $\mathcal{A}$ has constant budget, so all executions use one finite alphabet $\Gamma_B(q_*)$. For every pass index $i = 0, 1, \ldots, p$, the pair family consisting of the persistent store after pass $i$ and the tape $T_i(r)$ is pumpable.*

*Proof.* The base case is Definition S11.10. Apply Definition S11.11 successively to passes $1, \ldots, p$. □

**Remark S11.13** (Unary scratch tapes)**.** *If pass 1 writes one marker per input period, then $T_1(r) = \texttt{marker}^r$ is pumpable with live pump word* `marker`*. Definition S11.11 applies to every later pass. A later constant-budget pass may output one marker every $c$ input markers, permute markers, or add a fixed periodic decoration, but it remains pumpable. It cannot turn $\texttt{marker}^r$ into a family of $O(1)$ many correct aggregate circuits for all noncolliding $r$; if the final pump collapses, the final output repeats on a residue class.*

### S11.2.3 Collapse Versus Final-Output Length

**Lemma S11.14** (Pumpable outputs are bounded-or-linear on each residue)**.** *Let $W(r)$ be a pumpable word family with data $(h, D, A_\rho, B_\rho, C_\rho)$. For $r = h + \rho + kD$:*

*(a) if $B_\rho = \epsilon$, then $W(h + \rho + kD)$ is independent of $k$;*

*(b) if $B_\rho \neq \epsilon$, then*

$$|W(h + \rho + kD)| \geq k.$$

*Consequently, on a live residue,*

$$|W(r)| \geq \frac{r - h - D + 1}{D}$$

*for all $r \geq h + D$ in that residue.*

*Proof.* Both statements are immediate from $W(h + \rho + kD) = A_\rho B_\rho^k C_\rho$. If $B_\rho \neq \epsilon$, then $|B_\rho| \geq 1$, so $|A_\rho B_\rho^k C_\rho| \geq k$. Since $r = h + \rho + kD$ and $0 \leq \rho < D$, $k \geq (r - h - D + 1)/D$. □

**Lemma S11.15** (Correct noncolliding final outputs cannot collapse)**.** *Assume finite-range noncollision (Assumption S11.4) with $N(R) = R$ on the interval $1 \leq r \leq R$. Suppose the final output family $T_p(r)$ is pumpable with data $(h, D, A_\rho, B_\rho, C_\rho)$ and $\mathcal{A}$ is correct on every $r \in \{1, \ldots, R\}$. Without loss of generality take $h \geq 1$: if $h = 0$, replace $h$ by $h + D$; this preserves pumpability after discarding the finite prefix of indices $k = 0$ on each residue and absorbing one copy of $B_\rho$ into $A_\rho$. Then every residue class appearing twice in the tail $\{h, h+1, \ldots, R\}$ is live. In particular, for every $r \in \{h + D, \ldots, R\}$,*

$$|T_p(r)| \geq \frac{r - h - D + 1}{D}.$$

*Proof.* Let $r = h + \rho + kD$ with $k \geq 1$ and $r \leq R$. Then $r - D = h + \rho + (k - 1)D$ also lies in the tail, and by the convention $h \geq 1$ it satisfies $r - D \geq h \geq 1$, so both $r$ and $r - D$ lie in the correctness domain $\{1, \ldots, R\}$. If $B_\rho = \epsilon$, Definition S11.14 gives

$$T_p(r) = T_p(r - D)$$

as literal circuit strings. Hence the two outputs have the same unitary. Correctness at both $r$ and $r - D$ would imply

$$U(\mathcal{L}_B(C_r)) = U(\mathcal{L}_B(C_{r-D}))$$

up to global phase, which by exactness of the lowering means $D^r \equiv D^{r-D}$ up to global phase, contradicting Assumption S11.4 with $N(R) = R$. Therefore $B_\rho \neq \epsilon$, and Definition S11.14 gives the stated length bound. □

### S11.2.4 Final-Output Theorem and Corollaries

**Theorem S11.16** ($p$-pass final-output lower bound, restated with full hypotheses)**.** *Fix a fixed-width diagonal witness family $C_r = H_\Lambda D^r H_\Lambda^{-1}$ with exact lowering $\mathcal{L}_B(C_r) = \Pi M^r \Sigma$, and assume unbounded noncollision (Assumption S11.5): the powers $D^r$ are pairwise distinct up to global phase for all $r \geq 1$, so $N(R) = R$ on every range. Let $\mathcal{A}$ be a uniform deterministic $p$-pass finite-window, finite-burst transducer with constant budget that is correct on the input $\mathcal{L}_B(C_r) = \Pi M^r \Sigma$ for every $r \geq 1$. Here uniform means one fixed deterministic transducer, with the same passes, transition rules, finite state, window, burst bound, and store bound for all repetition counts $r$.*

*Then there exist constants $h \geq 1$ and $D \geq 1$, depending on $\mathcal{A}$ and the fixed witness but not on $r$, such that for every $r \geq h + D$,*

$$|T_p(r)| \geq \frac{r - h - D + 1}{D} = \Omega_{\mathcal{A}}(r).$$

*Equivalently, no constant-budget transducer in this model that is correct on the whole noncolliding fixed-width family can emit $o(r)$-length final outputs on that family.*

*Proof.* By constant budget, all executions use a finite charged token alphabet. Definition S11.12 gives pumpability of the final pair family and hence of $T_p(r)$, with data $(h, D)$ depending only on $\mathcal{A}$ and the witness. By the convention of Definition S11.15, take $h \geq 1$, so every comparison point $r - D \geq h \geq 1$ used below lies in the correctness domain.

Fix any $r \geq h + D$ and any $R \geq r$. By Assumption S11.5, $N(R) = R$, and $\mathcal{A}$ is correct on every $r' \in \{1, \ldots, R\}$. Thus Definition S11.15 applies and gives the displayed bound at $r$. In particular every final residue is live: if any final residue collapsed, two sufficiently large inputs in that residue would receive the same final output string, contradicting correctness and noncollision. □

**Correctness quantifier.** Correctness in Definition S11.16 is required at every $r \geq 1$; the conclusion then holds at every $r \geq h + D$, without exception. This global quantifier is not decorative: the collapse argument in Definition S11.15 compares the outputs at $r$ and $r - D$ and needs correctness at both points. Correctness at isolated points is not sufficient; if $\mathcal{A}$ were correct at $r$ but not at $r - D$, identical final strings at those two inputs would yield no contradiction.

**Corollary S11.17** (Finite-range tail form, restated with full hypotheses)**.** *Let $\mathcal{A}$ be a constant-budget transducer, and let $(h, D)$ be pumpability data for its final pair family as provided by Definition S11.12. Suppose finite-range noncollision (Assumption S11.4) holds with $N(R) = R$ on the range $1 \leq r \leq R$ and $\mathcal{A}$ is correct on every $r \in \{1, \ldots, R\}$. Then*

$$|T_p(r)| \geq \frac{r - h - D + 1}{D} \qquad \text{for every } h + D \leq r \leq R.$$

*The constants $(h, D)$ are determined by $\mathcal{A}$ and the witness family alone; in particular they do not depend on $R$. No statement is made for transducers outside the constant-budget regime: there, pumpability data in the sense of Definition S11.9 need not exist, and any finite-range inequality would require explicit control of pumping constants not supplied by this theorem.*

*Proof.* Constant budget puts all executions over one finite charged alphabet, so Definition S11.12 supplies pumpability data $(h, D)$ for the final tape. The construction of this data uses only the fixed transducer, its finite alphabets and state sets, and the fixed witness words $\Pi, M, \Sigma$; it is obtained before any finite endpoint $R$ is chosen. Hence $(h, D)$ are independent of $R$. Definition S11.15 then applies verbatim on the range $1 \leq r \leq R$. □

**Corollary S11.18** (Bit-level output counting toll, restated for proof)**.** *Assume finite-range noncollision supplies $N(R)$ pairwise distinct target unitaries, up to global phase, on a range. Suppose a deterministic exact compiler emits final outputs $T_p(r)$. Then*

$$\max_{1 \leq r \leq R} B_{\text{self}}(T_p(r)) \geq \lceil \log_2 N(R) \rceil .$$

*The same inequality holds for a fixed family-relative code when one public dictionary is shared by the whole range. For the rational-angle CP witness, only the displayed per-range inequality is claimed on fixed ranges $R \leq T = 131040$.*

*Proof.* A circuit string determines its unitary, so a deterministic exact compiler that is correct on two inputs whose target unitaries are distinct up to global phase must emit two distinct output strings. The noncollision hypothesis therefore forces at least $N(R)$ pairwise distinct codewords. If their prefix-free lengths are $\ell_1, \ldots, \ell_{N(R)}$, Kraft's inequality gives $\sum_i 2^{-\ell_i} \leq 1$, so $N(R) 2^{-\max_i \ell_i} \leq 1$. Hence the largest length is at least $\lceil \log_2 N(R) \rceil$. The argument applies independently to either fixed code and does not convert token count or gate count into bits. □

# S12 Additional experiments and protocols

## S12.1 Matched-representation benchmark

**Definition S12.1** (Matched-representation benchmark)**.** *For one sampled coefficient vector $x$, a matched cell fixes the target $U_x$, target basis, hardware model, total error budget, timeout, memory cap, and seed, and then evaluates every compiler on nine representations: (1) semantic aggregate, (2)*

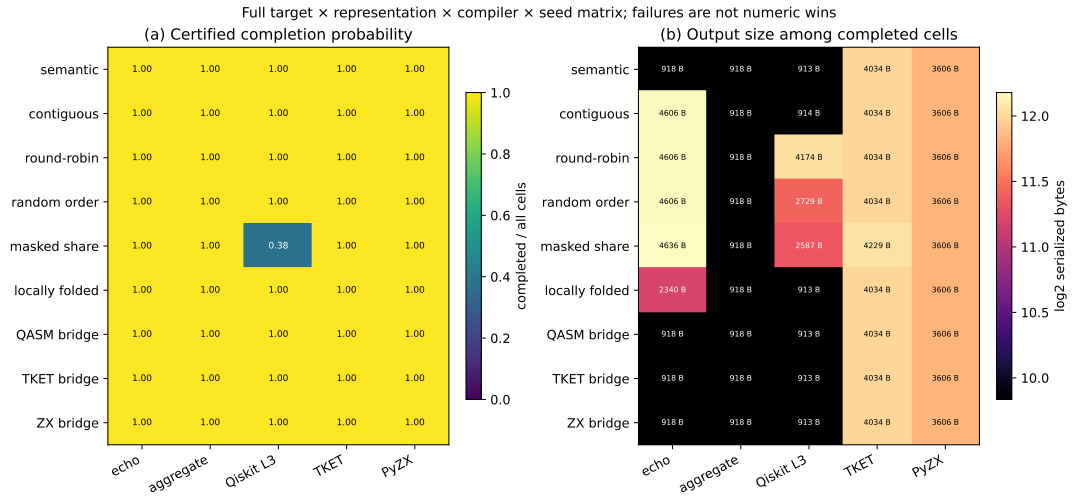



Figure S4: Complete matched-representation interaction summary. Panel (a) reports certified completion probability for every representation–compiler pair; panel (b) reports serialized output size only among completed cells. Predicate errors remain status observations and never become numerical wins.

*contiguous flat, (3) round-robin flat, (4) random commuting order, (5) masked-share/update stream, (6) locally folded flat, and (7–9) QASM, TKET, and ZX bridge records. Each representation has a unique identifier, serialized payload, generation seed, and unitary certificate. A cell is numerical only after the common certificate policy returns* `completed_valid`*; unsupported predicates and malformed or inconclusive records remain status outcomes.*

The executable campaign uses the no-wrap target

$$U_x = \prod_{j=1}^{m} \exp\left(-\frac{i\pi x_j}{8K} Z_j Z_{j+1}\right), \qquad x_j \in \{0, \ldots, K-1\}.$$

The active support is sampled at the requested density and each active coefficient is uniform on $\{1, \ldots, K-1\}$. The frozen campaign contains 1,080 matched cells across the nine representations and five compilers. Of these, 1,065 are `completed_valid`; the remaining 15 are retained as `predicate_error`, all from the Qiskit masked-update path, and are not converted into numerical wins or losses. Every raw row records the generation parameters, sampled $x$, realized density, schedule, basis, timeout, memory cap, runtime, peak RSS, gates, depth, CX, serialized input and output/IR bytes, bridge sequence, and certificate kind.

The representation main effect, compiler main effect, their interaction, completion probability, and scaling exponents are computed from the complete matrix. The interaction is substantive: a compiler that reconstructs a phase polynomial or Pauli network before synthesis belongs on the semantic-capable side even when its input arrived through a flat bridge.

**Lemma S12.2** (Dimension-preserving instrumentation)**.** *For every file $A$ named by an accepted version-one cut manifest (Definition S9.7), its reported size is exactly* $8\,\mathrm{stat}(A).$`st_size` *bits. Address growth is charged by the literal varint bytes; in particular* 127 *and* 128 *have different address lengths. There is no representation-independent conversion from gate count, depth,* `cx`*, token count, or IR-node count to bits.*

*Proof.* The first statement is Definition S9.7. Gate or node count does not determine parameter numerators, denominators, qubit-address widths, edge endpoints, delimiters, or textual external payloads. Thus equal-count objects can have different artifact lengths. Conversely, family-relative dictionary codes can assign different self-contained circuits equal conditional length. Only the named artifact file supplies a bit value. □

| Dimension | Recorded field | Executable serializer or definition | Permitted comparison |
|---|---|---|---|
| bits | `serialized_input_bits` | verified `SemanticStream`, `FlatUpdateStream`, or external-input artifact file | same codec/mode only |
| bits | `B_com_bits` | verified literal committed-output artifact/prefix file | same codec only |
| bits | five `B_*_bits` fields | verified control/store/window/parameter/IR files | corresponding formal field only |
| records/tokens | source records | exact count before serialization | never converted by a constant |
| IR nodes | `ir_nodes` | structural node count reported separately from `B_ir_bits` | no implicit bit or gate conversion |
| gates/depth/CX | structural output | Qiskit circuit operations, DAG depth, and `cx` count | like-for-like target basis |

Table S8: Instrumentation contract. Figures and tables compare within rows of one dimension. Cross-dimensional discussion cites the explicit codec or is stated only qualitatively.

| Evidence class | Included experiments | Authorized inference |
|---|---|---|
| instrumented theorem-model implementation | W3 traced reference compilers and restart verifier | directly tests the serialized commitment–cut coordinates and reports theorem residuals |
| configured-pipeline diagnostic | matched matrix, external semantic baselines, exact-semantics witnesses, and natural workloads | compares observed tool behavior; never proves a lower bound |
| model-specific consequence | fixed-total-error synthesis and surface-code calculation | reports consequences under the disclosed QRE; not a hardware-independent forecast |

Table S9: Formal/evidentiary separation used throughout the paper. The first row is the instrumented reference implementation of the declared resource model; configured external tools are not retroactively placed in that class.

## S12.2 TKET state-scaling scan (E7)

The stance of this campaign is explanatory, not competitive. TKET PauliSimp and PyZX full-reduce already implement the semantic-first principle on their global IRs; the contribution of this paper is the bound that says what any such tool must pay to do so on a dispersed stream (Theorem 2 and Proposition 5). A preregistered scan probed this on a third-party compiler the authors do not control: with predictions and decision rule committed before execution (`PaperDraft/research/E7_preregistration.md`), TKET PauliSimp was run at $m \in \{8, 16, 32, 64, 128\}$, three seeds per size, with the TKET pass sequence DecomposeBoxes, AutoRebase, RemoveRedundancies, PauliSimp, RemoveRedundancies, AutoRebase, on the four-round masked modular-share streams of the benchmark generator (`masked_share_update` with $r = 4$ and $K = 8$: shares modulo $8K = 64$ on the chain characters $Z_j Z_{j+1}$, every share nonzero and none equal to the target coefficient). These are dispersed streams in the sense of Definition 1 at a fixed modulus; they are not the two-round $Q = 3$ masked-share device of Theorem 3. In every cell PauliSimp reduced the non-Clifford rotations to one per active term (508 dispersed rotations to 127 non-Clifford $R_z$ at $m = 128$), although its output was not compact in size: at $m = 128$ it emitted 688–705 $R_z$ gates in total and 5.3 times the input QASM text. Its process memory (mean RSS increment) grew monotonically in $m$, with log–log slope 1.70 over $m \in \{32, 64, 128\}$, the range selected by the preregistered noise-floor rule (1.14 over all five sizes), while the instrumented semantic-input control stayed flat (196 → 199 bytes over the same 16× range of $m$). The preregistered prediction was monotone growth with log–log slope in $[0.7, 1.5]$, evaluated on $m \in \{32, 64, 128\}$ when the small sizes fall below the noise floor; it was thereby rejected under its own frozen decision rule, and we report it as rejected, noting that the all-size slope would have fallen inside the interval. The frozen artifact is `data/frozen/e7_tket_state_scaling.json`. Two limits on what this scan is

allowed to establish should be stated explicitly. First, the quantity measured at a third-party API boundary is a process-memory increment, which is a configured-pipeline diagnostic in the sense of Table S9: it is not promoted to a measurement of $B_{\mathrm{cut}}$ and is not commensurable with the $S$ of the frontier. The scan also has no arm that runs PauliSimp on the aggregated layer, and its input text itself grows linearly with $m$, so it does not isolate a cost of dispersal: it describes PauliSimp's behavior on dispersed streams, while the instrumented control shows only that the authors' semantic-input compiler stays flat. It tests neither the constant nor the exponent of the frontier. Second, because the measured quantity is not $B_{\mathrm{cut}}$, the scan cannot say how close this tool comes to the floor; growth faster than linear over the evaluated range is compatible with implementation structure (the pairwise commutation structure of a Pauli graph is itself superlinear in the number of terms). No claim that the authors' implementation outperforms these tools is made or implied.

## S12.3 Third-party commitment-toll attempts (E8b)

Two attempts to carry the toll onto compilers we do not control were preregistered, and **both failed**; we report them rather than repair them. The first asked only whether real toolchains spend committed bytes on addresses, and its design placed every rotation on the low-index qubits, so it never varied the address width it was meant to vary (a corrected rerun, labelled non-preregistered in the frozen artifact, finds 2.95 bytes of growth per rotation over a 1000× range of register width, itself below the preregistered threshold). The second tested the mechanism directly: two circuits identical in $m$, in $k$, in rotation count and in angle multiset, differing only in whether the dispersed subset is an arithmetic progression or a random $k$-subset, emitted through Qiskit and TKET and compared by compressed size. Its textual control passed—the variants differ by under 0.05% of raw bytes, so the comparison is not confounded by text length—and on TKET the excess reaches 86% of $\log_2 \binom{m}{k}$ at $k = m/2$, comfortably above the seed noise. But the excess peaks at $k = m/4$ rather than $k = m/2$, and on Qiskit it is 8% of prediction and smaller than the seed-to-seed noise. Under the frozen rule this is a failure, and we make no third-party claim. Two causes are possible and the run does not separate them: a general-purpose compressor applied to a 100 kB instruction text may be too coarse an entropy estimator for a few hundred bytes of schedule information, and neither tool emits a round-major committed stream (Qiskit's OpenQASM 3 output is ordered by qubit, and TKET's lists the full first layer before the dispersed addresses), so the design premise that the output is a committed prefix was not met. The record-level toll of Proposition 7 rests on the source-coding argument and on the instrumented device of Fig. 3; whether it is measurable at a third-party API boundary remains open.

## S12.4 Packing-Family Scaling

The complete multifactor campaign of Fig. S4 is the primary matched-representation experiment. The earlier 5-by-5 sweep below is retained as a separately keyed legacy diagnostic because it reaches the same maximum width under a 600 s protocol and preserves its historical `predicate_error` outcomes; it is not substituted for the new matrix. This legacy experiment links the growing-$m$ theorems to configured-pipeline behavior. Unlike the fixed-width witnesses below, which exercise the exact-semantics corollary, the packing family grows the number of independent phase terms $m$ at fixed tolerance.

**Instance.** We instantiate the no-wrap packing $U_x = \prod_j R_{P_j}(\Delta x_j)$, $x \in [K]_0^m$, $(K-1)\Delta \le \pi$, with the chain of two-body characters $a_j = e_j + e_{j+1}$ on $n = m+1$ qubits, which are $\mathbb{F}_2$-independent for every $m$ (certified by rerunning the exact rank verifier), using the separate experimental setting $K = 2$, $\Delta = \pi/4$. Each token is lowered as $RZ_i(\theta)\, RZ_j(\theta)\, CP_{ij}(-2\theta)$ under the convention used here. The balanced round-robin encoding, in which each of the $r$ rounds carries the residue $x_j/r$ on the schedule $\sigma(i) = ((i-1) \bmod m) + 1$, with dispersal $r = 4$ interleaves the terms across shared wires, so per-wire locality does not trivially reveal the aggregate. Because these balanced tokens contain $x_j/r$, this is a pipeline diagnostic rather than an instance of the masked-share lower bound. The sweep is $m \in \{4, 8, 16, 32, 64\}$ under the standard 600 s protocol of this section. The exact rational-$\pi$ sidecar records $\Delta/\pi = 1/4$ and per-token angle $1/16$. Correctness uses the dense

| $m$ | semantic UCC | `qiskit opt3` | TKET PauliSimp (rebased) | staq folding | phase-poly ref |
|---|---|---|---|---|---|
| 4 | 25/21/8 | 68/38/32 | 54/20/10 | 40/27/32 | 25/21/8 |
| 8 | 49/41/16 | 136/54/64 | `predicate_error` | 80/39/64 | 49/41/16 |
| 16 | 97/81/32 | 272/86/128 | `predicate_error` | 160/63/128 | 97/81/32 |
| 32 | 193/161/64 | 544/150/256 | `predicate_error` | 320/111/256 | 193/161/64 |
| 64 | 385/321/128 | 1088/278/512 | `predicate_error` | 640/207/512 | 385/321/128 |

Table S10: Packing-family scaling at growing $m$ (chain two-body characters, $n = m + 1$, dispersal $r = 4$, 600 s budget). Completed cells report output gates/depth/`cx`; status outcomes are recorded separately and are not scored as structural wins. No external row is claimed to be a member of the transducer class of Definition S9.3.

certificate for $n \leq 6$ and the sound exact symbolic checker of Definitions S10.1 and S10.2 for $n > 6$; any output outside $\mathcal{D}_{\mathrm{CPC}}$ is status-only.

**Results.** Table S10 reports completed output gates as a function of $m$ for semantic UCC, `qiskit opt3`, rebased TKET PauliSimp, staq rotation folding, and the phase-polynomial reference. All five paths completed at $m = 4$; semantic UCC, `qiskit opt3`, staq rotation folding, and the phase-polynomial reference also completed at $m = 8, 16, 32, 64$, while rebased TKET PauliSimp returned `predicate_error` at those four sizes because its output fell outside the Hadamard-free symbolic-certificate predicate. Semantic UCC and the phase-polynomial reference produced the same linear resource sequence, from 25/21/8 gates/depth/`cx` at $m = 4$ to 385/321/128 at $m = 64$. Among the completed external rows, `qiskit opt3` and staq produced $17m$ and $10m$ output gates, respectively, as the $rm$ token stream grew. Status outcomes are recorded separately and omitted from numerical curves.

The earlier two-panel figure is deliberately omitted. Its gate panel is redundant with Table S10, while its family-relative bit panel was generated from a handwritten length formula and has no serialized per-point files. Under the present accounting contract such a curve may be restored only by emitting the exact conditional-code artifacts (together with the family-dictionary digest) and measuring their files.

**Scope.** The external-toolchain rows are configured-pipeline diagnostics in the sense of this section: they neither prove nor test Theorem 3, whose serialized inter-round cut budget is not observable from output gate counts, and no bit curve is inferred from these gate-count rows. The separate reference-compiler implementation below measures theorem-model cut fields directly; it is not inferred from output gates.

## S12.5 Event-Level Commitment–Cut Tracing

The instrumented suite contains echo/pass-through, full aggregation, 25%, 50%, and 75% memory-capped hybrids, and an IR-materializing control. Each compiler follows the formal flat-update interface and records an event after every input update and commitment. The campaign scans $m, r, K, \epsilon, p, S$, schedule, and seed, producing 592 complete run summaries plus the compressed raw event log and content-addressed trace archive. At each named cut it records input position, pass, control/store/window/parameter and IR bytes, committed-output bytes, current output length, RSS, and wall time. The literal measured definitions are

$$B_{\mathrm{com}}(c) = 8|A_{\mathrm{com,c}}|_{\mathrm{byte}},$$
$$B_{\mathrm{cross}}(c) = 8 \sum_{f \neq \mathrm{com}} |A_{f,c}|_{\mathrm{byte}}.$$

RSS is recorded as an implementation metric and is not substituted for the serialized state cap $S$. The analyzer recomputes the theorem residual from the trace coordinates and retains every anomaly or failed run. All frozen points lie in the theorem-allowed region; no point was removed as an outlier.

Echo commits updates early with little retained state; full aggregation occupies the state-heavy compact-output corner; and the three capped hybrids interpolate through explicit block

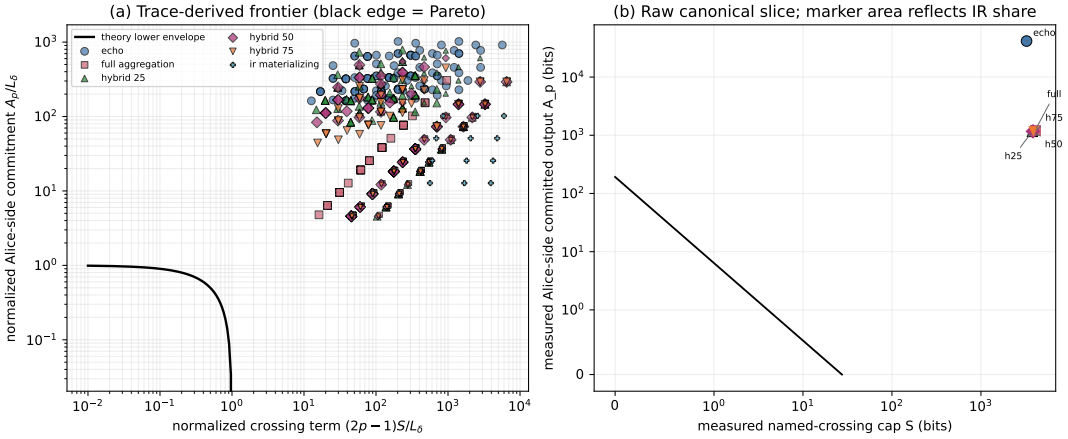



Figure S5: Trace-derived commitment–crossing-state–IR surface. The black edge marks measured Pareto points and the theoretical lower envelope is evaluated in the same serialized-bit coordinates. Marker area in the raw slice reflects IR share; RSS is recorded separately.

commitments. Their scaling fits and residuals are reported with the raw points. The measured quantities are the theorem's resources in its own bit units, but at these sizes they are dominated by a fixed serializer overhead of about 3.8 kbit, against a bound of 10–180 bits; only the increments with $m$ are informative, and the trace is a consistency check of the accounting, not a test that could approach the bound.

## S12.6 Width-Axis Scaling

To check that the semantic path emits $O(m)$ phase records independently of the repetition count (Section VI), we run a width-axis sweep for the `full_pair_cp` topology at $n = 4, 5, 6$.

| Width ($n$) | CP pairs ($m_{\mathrm{CP}}$) | Output Gates | CX Count |
|---|---|---|---|
| 4 | 6 | 42 | 12 |
| 5 | 10 | 65 | 20 |
| 6 | 15 | 93 | 30 |

Table S11: Width-axis scaling for the semantic Fourier-layer witness. For each fixed width $n$, the semantic output is constant across all tested repetition counts $r$. Across widths, the output scales as $O(m_{\mathrm{CP}})$ where $m_{\mathrm{CP}} = n(n-1)/2$.

The output is $O(m_{\mathrm{CP}})$ and independent of $r$ on this finite width sweep, consistent with that property.

## S12.7 Exact-semantics witness results

These experiments exercise the exact-semantics corollary (Definition S11.6), not the main resource tradeoff (Theorem 3). They verify the constructive aggregation path on the tested fixed-width witnesses and record how configured external pipelines behave predictably across the representation boundary. Full data tables are in Section S14.

The rational CP $HD^rH$ witness is the canonical fixed-width family with a constant 42-gate output. Figure 4 shows that semantic UCC remains at 42 gates while rebased TKET PauliSimp returns 52–73 gates across five scales; materialize-first baselines grow linearly. The irrational independent-Pauli witness (Fig. S6) shows the same pattern at 27-gate constant output. Key observations:

- Semantic UCC outputs a constant gate count (42 for rational, 27 for irrational) at all five tested sizes.

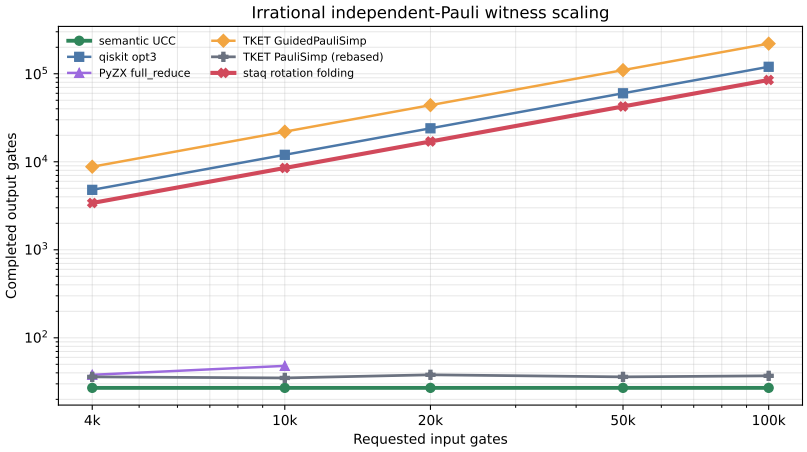



Figure S6: Configured-pipeline scaling on the irrational independent-Pauli witness. Semantic UCC remains at 27 gates. Rebased TKET PauliSimp returns a small validated form at all five scales, demonstrating that a generic pipeline can cross the formal boundary by reconstructing an aggregate Pauli representation. Public staq rotation folding and `qiskit opt3` grow with input size; TKET GuidedPauliSimp, which acts only inside `CircBox` regions, does not optimize these flat inputs. PyZX `full_reduce` completes at 4k and 10k, fails the numerical tolerance at 20k, and reaches the time budget at larger scales. Status-only cells are omitted from numerical curves.

| Requested | semantic UCC | artifact UCC (Fourier disabled) | `qiskit` opt3 | phase-poly ref |
|---|---|---|---|---|
| 4,000 | 42 / 2.9 s | 9,588 / 18.2 s | 9,588 / 1.0 s | 42 / 0.2 s |
| 10,000 | 42 / 8.7 s | 23,988 / 94.2 s | 23,988 / 5.1 s | 42 / 0.3 s |
| 20,000 | 42 / 28.5 s | 47,988 / 339.3 s | 47,988 / 19.5 s | 42 / 0.3 s |
| 50,000 | 42 / 20.7 s | timeout | 119,988 / 120.9 s | 42 / 0.5 s |
| 100,000 | 42 / 28.7 s | timeout | 239,988 / 485.8 s | 42 / 0.8 s |

Table S12: Fourier-layer semantic-path ablation on the fixed-width $HD^rH$ witness under the 600 s budget. Disabling the Fourier-layer IR completes at 4k–20k but materializes large outputs; 50k–100k exceed the budget. Fourier-enabled semantic UCC and the phase-polynomial reference return 42 gates at all sizes.

- Rebased TKET PauliSimp recovers a small validated form at all five scales on both witnesses, demonstrating that a generic pipeline can cross the formal boundary by reconstructing an aggregate Pauli representation.
- Public staq rotation folding and `qiskit opt3` grow linearly with input size. TKET GuidedPauliSimp returns exactly the gate count of its lowered input at every size: it applies PauliSimp only inside `CircBox` regions, which these flat inputs do not contain, so it is not an optimizing baseline here.
- PyZX `full_reduce` completes at small sizes but reaches time or correctness limits at larger scales.

These measurements exercise the constructive side of Definition S11.6 and disclose the behavior of configured tools; they do not prove the resource bound for any real compiler. The rebased PauliSimp result is the classification's positive prediction in action: once a generic pass sequence reconstructs a compact Pauli representation, the output need not grow with the flat input. Conversely, staq's growing output and PyZX's status outcomes are not impossibility results—they are diagnostics of configured-pipeline behavior on this specific witness family.

## S12.8 Mechanism Ablation

The ablation strengthens the mechanism interpretation. With Fourier-layer IR disabled, the compiler completes at 4k, 10k, and 20k but materializes the same large non-aggregated outputs as

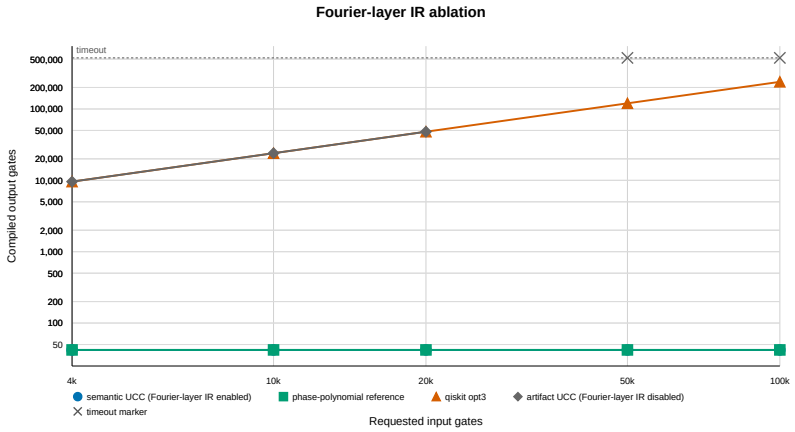



Figure S7: Mechanism ablation for the fixed-width $HD^rH$ witness. Disabling the Fourier-layer semantic path yields completed but linearly large outputs through 20k and timeout rows at 50k and 100k, while Fourier-enabled semantic UCC and the phase-polynomial reference recover the canonical 42-gate circuit.

`qiskit opt3`; at 50k and 100k the disabled-IR rows exceed the 600 s budget (Fig. S7). In contrast, the Fourier-enabled semantic UCC path and the phase-polynomial reference recover the same 42-gate canonical circuit at every tested size. Thus the constant form is tied to semantic Fourier-layer lifting and coefficient aggregation, not merely to a favorable ordering of flat post-lowering passes.

## S12.9 Strong baselines and the real-task panel

The canonical comparison includes Qiskit `optimization_level=3` [4], TKET PauliSimp [5, 6], PyZX `full_reduce` [7], and the authors' exact phase-table reference. Each external tool has a recommended-native and a unified $\{\mathrm{CX}, R_Z, H\}$ configuration, with pass sequence, bridge, version, seed, timeout, memory cap, and certificate frozen. TKET supplies the Pauli-network/global-semantic class and PyZX the graph/global-IR class. The complete W4 interaction is Fig. S4; the independent W5 campaign and bridge-loss accounting are described in Section VI. Historical staq, GuidedPauliSimp, and phase-polynomial diagnostic rows remain in their original immutable campaigns and are not silently mixed into the canonical matrix. This comparison is diagnostic, not a contest: the external tools are treated as independent implementations of the semantic-first principle whose internal costs the bounds of Sections III and IV explain, gate-count parity is reported as parity, and no headline ranks the authors' pipeline above them.

| Instance | Method | Gates | Depth | CX | Runtime |
|---|---|---|---|---|---|
| `REAL24-phase-est.` | translation only | 159,838 | 121,214 | 67,608 | 0.080 s |
| | qiskit opt3 | 166,805 | 119,204 | 58,491 | 1.298 s |
| | upstream UCC v0.4.12 default | 471,819 | 328,271 | 58,491 | 80.923 s |
| | semantic UCC | 166,805 | 119,204 | 58,491 | 1.364 s |
| `REAL24-grover` | translation only | 85,403 | 55,892 | 33,408 | 0.104 s |
| | qiskit opt3 | 79,029 | 54,163 | 32,544 | 0.552 s |
| | upstream UCC v0.4.12 default | 287,047 | 152,298 | 32,544 | 1.915 s |
| | semantic UCC | 79,029 | 54,163 | 32,544 | 0.556 s |
| `REAL24-qaoa` | translation only | 36,512 | 2,416 | 23,808 | 0.022 s |
| | qiskit opt3 | 36,512 | 2,416 | 23,808 | 0.272 s |
| | upstream UCC v0.4.12 default | 167,833 | 7,133 | 23,808 | 1.010 s |
| | semantic UCC | 36,512 | 2,416 | 23,808 | 0.273 s |

Table S13: Official real algorithm-family instances. The upstream UCC v0.4.12 row is the frozen pre-semantic default defined in the provenance map of Section S13. Parity with `qiskit opt3` reflects conservative portfolio selection; this panel is an anti-regression check, not evidence of semantic recovery.

These results support the recoverability-parity regime (Fig. S8): the semantic artifact matches `qiskit opt3`-level output quality while improving over the upstream UCC v0.4.12 pre-semantic

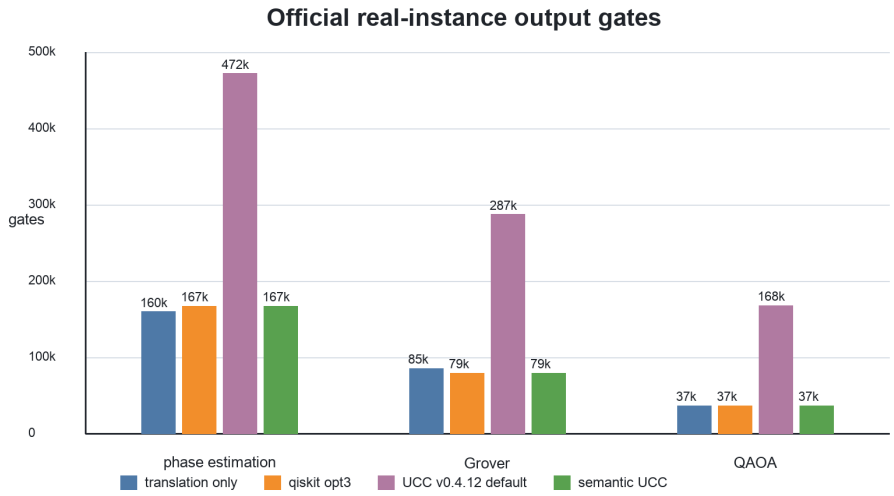



Figure S8: Total output gate counts on the three official real instances. The semantic UCC artifact matches `qiskit` opt3-level output quality while substantially improving over the upstream UCC v0.4.12 pre-semantic default.

default of Table S13 by approximately 65%, 73%, and 78% gate reduction on the three instances. The selector cost in this panel is the implementation's conservative structural score $\kappa = \text{gates} + \text{depth} + 10 \cdot \text{multi-qubit}$, with candidate-control guards; the factor 10 is a conservative lexicographic-style heuristic favoring lower two-qubit exposure, used only for portfolio selection. This explains the `REAL24-phase-est.` row (source key `phase_estimation_real`) in which translation-only has fewer total gates than the selected candidate but more `cx` gates and a higher score (957,132 versus 870,919). This quality-recovery pattern shows that semantic lifting behaves as an anti-regression mechanism on workloads beyond the minimal Fourier construction.

### S12.10 Natural-workload figures

## S13 Artifact Engineering

The implementation described in this section is a bounded recognizer and candidate controller, not a general maximal-commuting-region extractor for arbitrary DAGs. Its semantic nodes are limited to explicit supported Pauli/phase records, local phase-ladder or Fourier spans, mirrored/self-inverse composite blocks, and repeated-run or preset semantic references. The word "region" below means a contiguous span accepted by that configured grammar under its scan order. Soundness of a selected rewrite is supplied by the independent certificate and conservative fallback, not by a claim of global maximality.

### S13.1 Candidate Control and Bounded Pipeline

The semantic path is advisory, not authoritative: semantic terms are lowered late, semantic subterms are compiled once and reused, and a semantic candidate loses to a flat reference whenever it does not clearly improve the conservative structural cost. The term caches, compiled-subterm caches, and repeated-run block caches in the implementation follow from treating the semantic term rather than the materialized circuit as the unit of comparison. The whole method is the following bounded pipeline.

1. **Input.** A high-level circuit $C$, target basis $B$, and iteration budget $K$.
2. Compute stable signatures for instructions and candidate blocks in $C$.

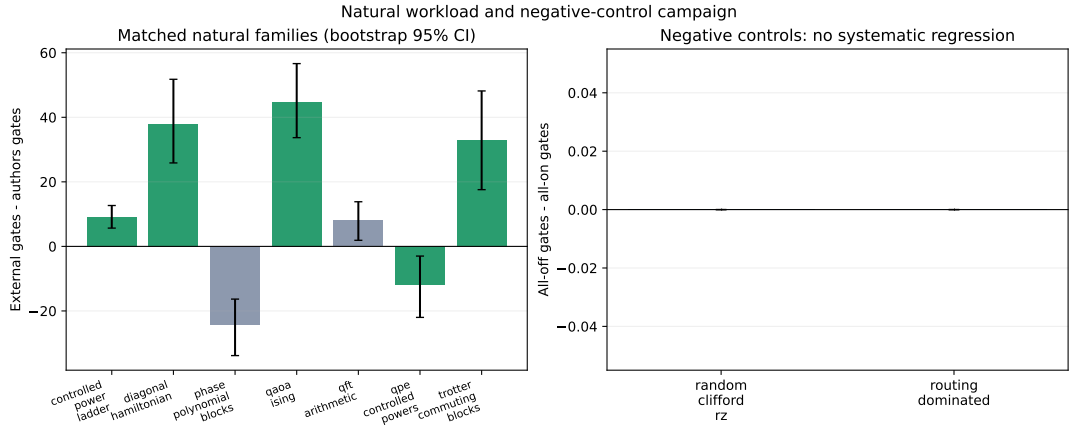



Figure S9: Matched natural-family comparison and negative controls. Error bars are paired bootstrap 95% intervals over matched cells; because the generators of several families do not depend on the seed, these intervals describe the spread over cells, not seed replication. All structured families and both controls are retained.

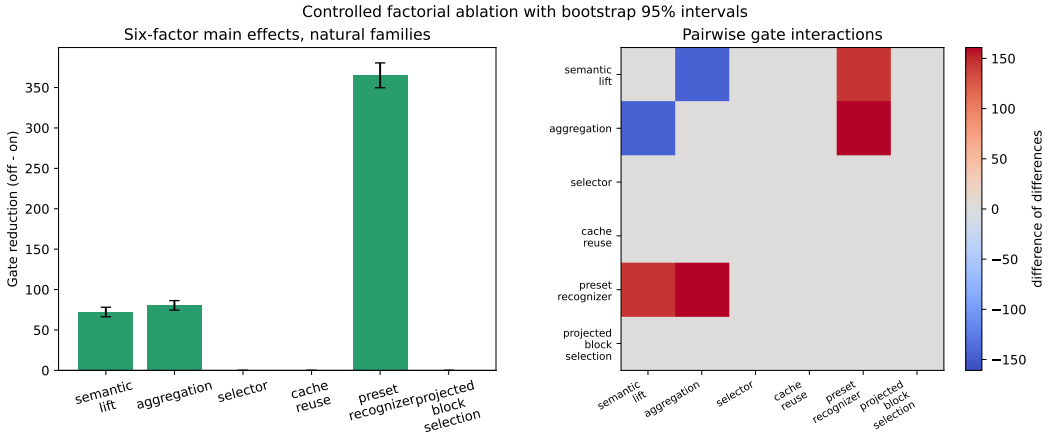



Figure S10: All six factorial gate-count main effects and pairwise interactions. Zero-effect factors are displayed rather than omitted.

3. Run bounded preprocessing $\mathcal{S}_K(C)$: repeated-prefix and repeated-run detection and reuse; adjacent block–inverse(block) cancellation; commutative inverse cancellation; semantic lifting into instruction-span, phase-ladder, and mirrored/self-inverse nodes where applicable; early stop if one full round does not improve the selected cost.

4. Construct the candidate set

$$\mathcal{C}(C) = \{\mathcal{L}_B(C), \mathcal{P}_{\mathrm{ucc}}(\mathcal{L}_B(C)), \\ \mathcal{P}_{\mathrm{ucc}}(\mathcal{L}_B(\mathcal{S}_K(C))), \mathcal{R}(C)\},$$

   where $\mathcal{R}(C)$ is an optional semantic or preset-reference candidate for large repeated structures, itself produced from the semantic term algebra (Fourier-layer lowering, mirrored/self-inverse lowering, or a bounded repeated-run rebuild).

5. If the circuit factors as $C = P \cdot B^r \cdot S$, form the bounded block-candidate set $\mathcal{B}(B) = \{B_1, \ldots, B_q\}$ and select $i^\star = \arg\min_i \kappa_{\mathrm{proj}}(P, B_i, r, S)$ on the projected full-circuit cost, rebuilding only $B_{i^\star}$. This is what lets the repeated-run path avoid materializing several giant candidates merely to compare them.

6. Evaluate all candidates under the conservative structural cost $\kappa$, with fixed lexicographic tie-breaking, and return $\widehat{C} = \arg\min_{X \in \mathcal{C}(C)} \kappa(X)$.

The role of candidate control is not to dominate all downstream flows. It is to prevent avoidable structured-circuit regressions by preserving the best available output among a small family of semantically equivalent candidates.

## S13.2 Complexity Discussion

The method is intentionally bounded and does not attempt global optimization. Let $n := |C|$ be the number of high-level instructions. Suppose one outer iteration consists of $r$ bounded scans, and scan $j$ has cost $T_j(n)$. Then the total runtime of preprocessing satisfies

$$T_{\text{pre}}(n) \in O\left(K \cdot \sum_{j=1}^{r} T_j(n)\right).$$

The dominant scans are near-linear in $n$ up to implementation-dependent hashing, block-comparison, and bookkeeping costs, so practical behavior stays near-linear up to a bounded constant on repeat-heavy workloads. A large-circuit fast path handles repeat-dominant cases, source-level short-circuiting avoids downstream work on Qiskit-native all-to-all composites, and backend-aware portfolio expansion is restricted to cases where instability matters. The method is therefore a bounded systems heuristic, not a globally optimizing pass.

**Proposition S13.1** (Expected regime of benefit)**.** *The preprocessing stage is most likely to help on circuits whose useful structure is already explicit at the high-level instruction stream or in the bounded semantic lift, especially exact inverse blocks, repeated blocks, phase-ladder layers, mirrored/self-inverse shells, and dominant repeated prefixes. It is not expected to uniformly improve families whose performance is dominated by later synthesis or routing decisions that are not captured by the preprocessing signatures or semantic nodes.*

*Justification.* This follows from the construction of the pass. Every preprocessing rule operates on high-level structural regularities detected through stable signatures, exact inverse relations, or the bounded semantic lift into semantic terms such as Fourier layers and mirrored/self-inverse shells. Therefore the pass can only exploit structure that survives in this representation. Conversely, if output quality is dominated by later basis-specific synthesis or hardware-routing effects not visible at this stage, the preprocessing pass may improve little or only preserve parity with stronger downstream baselines. The representation-aware compilation principle is therefore selective rather than universal: it claims that some opportunities are materially easier to preserve and compare before lowering, not that all useful optimization should be moved to the semantic layer. □

## S13.3 Integration Into UCC

The semantic artifact changes the UCC flow in six places:

1. it performs structural simplification before basis lowering,
2. it lifts suitable subcircuits into a bounded semantic term algebra before candidate generation,
3. it compares candidate outputs conservatively to avoid structured-circuit regressions,
4. it uses source-level short-circuiting on suitable all-to-all Qiskit-native cases,
5. it restricts backend-aware portfolio expansion to cases where extra probing is justified,
6. it memoizes bounded semantic references and repeated-run block selections for reuse.

The organizing distinction is that some circuit growth is unavoidable basis-translation overhead while some is structural regression caused by losing recoverable semantic evidence before candidate selection.

| Paper name | JSON key | Tool/pass sequence | Artifact flags | Budget | Source classes |
|---|---|---|---|---|---|
| semantic UCC (Fourier-layer IR enabled) | semantic_ucc | ucc.compile to {cx, rx, ry, rz, h} with Fourier-layer IR enabled | UCC_DISABLE_FOURIER_LAYER_IR unset | 600 s | main, abl, res, corr |
| upstream UCC v0.4.12 pre-semantic default | baseline_ucc | ucc.compile loaded from the legacy baseline repository in research/compare_real_instances.py; target basis {cx, rx, ry, rz, h} | legacy baseline repo; semantic candidate controls absent | 240 s in real-panel runner | real, appx |
| artifact UCC (Fourier-layer IR disabled) | no_fourier_ucc | same artifact compile path with Fourier-layer IR disabled | UCC_DISABLE_FOURIER_LAYER_IR=1 | 600 s | abl, res |
| qiskit opt3 | qiskit_opt3 | qiskit.transpile(...,optimization_level=3,layout_method="trivial",routing_method="none") | none | 600 s | main, abl, res |
| PyZX configured bridge | pyzx_configured | Qiskit QASM2 to PyZX, to_basic_gates, basic_optimization, Qiskit target-basis cleanup | none | 600 s | main |
| PyZX full_reduce | pyzx_full_reduce | Qiskit QASM2 to PyZX graph, full_reduce, circuit extraction, Qiskit target-basis cleanup | none | 600 s | main |
| TKET FullPeephole | tket_fullpeephole | Qiskit-to-TKET bridge when available, otherwise QASM fallback, then FullPeepholeOptimise | none | 600 s | main |
| TKET PauliSimp (unrebased configuration) | tket_paulisimp_unrebased | direct Qiskit–TKET bridge, then PauliSimp; retained only to record the predicate error | none | 600 s | status |
| TKET PauliSimp (rebased) | tket_paulisimp_rebased | direct bridge, DecomposeBoxes, AutoRebase({CX, RZ, Rx}), RemoveRedundancies, PauliSimp, RemoveRedundancies, target-basis cleanup | generic identical preamble | 600 s | main, irr |
| TKET GuidedPauliSimp | tket_guided_paulisimp | same TKET bridge/fallback sequence, then GuidedPauliSimp | none | 600 s | main |
| staq rotation folding | staq_rotation_folding | Qiskit opt0 target-basis QASM2, public staq fold_rotations, QASM2 parse, target-basis cleanup | no family labels; generic double-precision parser/serializer patch disclosed | 600 s | main, irr |
| phase-polynomial reference | phase_poly_reference | recognize the $HD^rH$ witness, aggregate commuting rz/cp coefficients, then lower through Qiskit opt0 | none | 600 s | main, abl, res, corr |

Table S14: Method/provenance map for the canonical validation protocol. The direct Qiskit-to-TKET bridge was available in the validation environment; the QASM fallback is documented for reproducibility when that bridge is unavailable.

## S13.4 Method and Provenance Map

Every frozen numerical row is addressed by the complete primary key

$$(\texttt{experiment}, \texttt{instance}, \texttt{representation},\\ \texttt{method}, \texttt{seed}, \texttt{backend_hash},\\ \texttt{tool_version}, \texttt{artifact_commit}, \texttt{config_hash}).$$

The backend and configuration hashes are SHA256 digests of canonical JSON (sorted keys and no insignificant whitespace). A row collision on all nine fields is rejected. In particular, the reorder-probe hardware rows and the canonical hardware-aware rows have different experiment and configuration keys, and therefore different reader labels even when the underlying MQT instance name is the same. The frozen source is `research/experiment_registry/frozen_hardware_rows.json`; `scripts/generate_keyed_tables.py` validates key uniqueness and generates the three affected table fragments plus `expanded_primary_keys.json`. The generated labels expose the campaign, instance, seed, and the first eight configuration-hash digits. Hand-edited numeric rows are not accepted by this path.

The canonical validation data are grouped by experiment family in the artifact. The external-baseline method map immediately below uses the disclosed `timeout_s` = 600 second wall-clock budget per method per instance; it does not override the distinct `RP23`, `HW24`, or `REAL24` campaign manifests. The table maps paper-facing names to JSON keys, tool/pass sequences, artifact flags, and source classes. The older real-panel UCC default row is included only to define that frozen-control label in Section S14; it is not one of the canonical methods used in the 600 s external-baseline comparison.

The source classes in Table S14 expand as follows: Here "main" denotes membership in the canonical comparison campaign and does not necessarily imply inclusion in the two headline tables.

- main: canonical 600 s external-baseline comparison.
- abl: Fourier-layer IR ablation.
- res: resource-consequence check.
- corr: large-instance correctness extension.
- irr: irrational independent-Pauli unbounded witness suite.

- status: provenance-only configuration excluded from numerical quality comparison when its predicate fails.
- real: official real-instance frozen panel.
- appx: older frozen-control appendix panels.

The new executable protocol and provenance are in `research/round6_major_revision/`: the witness declaration and exact-rank verifier, `unbounded_witness_results.json`, `rational_cp_round6_results.json`, `public_phase_folding_results.json`, `tket_paulisimp_results.json`, per-cell logs, and the environment snapshot. After global-phase alignment, the implemented check computes the maximum elementwise error $\epsilon$ and requires $\epsilon \leq \text{atol} + \text{rtol} \max_{i,j} |U_{ij}|$. Thus atol = rtol = $10^{-9}$ gives an effective upper threshold of approximately $2\times10^{-9}$ because unitary entries have magnitude at most one. Recorded maximum errors increase with repetition count $r$, while every row marked completed and valid satisfies this criterion. The irrational-witness PyZX 10k result passes at approximately $1.05 \times 10^{-9}$; the 20k result at approximately $2.68 \times 10^{-9}$ is a correctness failure. Timeout, predicate-error, unsupported-conversion, and correctness-failure statuses are kept distinct. The staq provenance records public commit `a2acd39` (with the full hash in the artifact) and the generic double-precision OpenQASM patch used for continuous-angle round trips.

## S13.5 Certificate, Cut-Trace, and QRE Artifacts

The exact canonicalizer and symbolic checker are `certificates/canonicalize.py` and `certificates/symbolic_checker.py`. The frozen audit `data/frozen/certificate_audit.json` contains 12 accepted/rejected dense cross-check cases over widths one through six and 56 adversarial mutations. The checker and dense evaluator use separate implementations: the former propagates complete signed Clifford tableaus and exact rational-$\pi$ Pauli coefficients, while the latter constructs full numerical matrices and a projective distance. Only `completed_valid` records carrying an independent certificate enter numerical means; unsupported, predicate-error, and numerical-inconclusive records remain status-only.

The canonical W3 theorem-model reference compilers are under `compiler/reference_streaming/` and are driven by `scripts/run_tradeoff_sweep.py`. After every input update, their version-one event schema records `committed_output_bytes`, the five cut fields from (S3), pass index, RSS, elapsed nanoseconds, artifact paths, and digests. The verifier reopens and decodes every field slice and checks the committed-prefix hashes. The complete 592-run campaign is stored under `data/runs/w3-tradeoff-20260802-local/`; its registered summary and freeze manifest are `data/frozen/tradeoff_summary.csv` and `data/frozen/tradeoff_freeze_manifest.json`. The older `research/cut_trace/` bundle was produced by an earlier serializer and lacks the per-event field files, so its 1,320 events and 24 summaries are historical only and are not accepted by the version-one verifier. The checked three-cut codec fixture and its manifests remain in the project-root `instrumentation/example_trace/`.

The W7 runner and allocator are the project-root `scripts/run_qre_campaign.py` and `scripts/allocate_synthesis_error.py`; their total-error schema, two QEC profiles, and factory grid are under `qre/`. The 288-row Parquet/CSV, full angle records, analysis, figures, and manifests are under `data/frozen/`, `data/runs/w7-fixed-total-error-20260803-local/`, and `figures/`. The earlier single-profile runner and its six-row result are retained as historical artifacts under `research/fixed_budget_qre/` but are not the W7 main result. The grid synthesizer is built from public staq commit `a2acd39e60ed0e5f1978fa530df7e1e0c7cedf7a`; the disclosed artifact change only replaces the nondeterministic candidate-search seed with 20260801. The runner omits staq's `-time` fast path because that path bypasses its internal detail/check branch; it instead measures subprocess time externally and requires `Check flag = 1` for every unique angle. The QRE is the explicit surface-code model family stated in the main text, not a vendor service or an independent architecture implementation.

## S13.6 Immutable Campaign Hierarchy

The machine-readable index `research/campaign_manifests/campaign_index.json` defines two base profiles and five noninterchangeable campaigns. `RP23` is the pass-order probe, `HW24` is the

| ID | claim location | primary evidence | authorized scope |
|---|---|---|---|
| C01 | abstract/introduction | Definitions 2 and S9.3 | formal model only |
| C02 | abstract/introduction | Theorem 2 | round-major forward scans re-delivered by the source; the $p$ forward crossings charged |
| C03 | introduction | Theorem 3 | deterministic one-pass zero-error compilers; $K \in \{Q-1, Q\}$ |
| C04 | abstract/introduction | Propositions 2 and 3 | family-relative codes; leading order for $p\log(mQrp) = o(m\log Q)$ |
| C05 | introduction | Propositions 4 and 5 | coefficient-freedom hypothesis; rounds contiguous up to generator-permuting Clifford records |
| C06 | abstract/introduction | Proposition 7 | record-level executed commitment; mask-uniform compilers |
| C07 | abstract/introduction | Corollary 2 | unitary-level executed commitment; proved, not measured |
| C08 | introduction | Figs. 2 and 3 | instrumented masked-share device; constructive checks |
| C09 | validation | Figs. S4 and 4 | configured-pipeline diagnostics |
| C10 | resource consequences | Table I and Fig. 5 | one disclosed surface-code model; inputs differ between pipelines |
| C11 | application workloads | Table II and Fig. S10 | 9 families; six-qubit dense certificates |
| C12 | compiler | Definition S10.2 | declared symbolic domain only |

Table S15: Claim–evidence matrix for the main text. A claim outside the displayed scope is not made by the paper.

canonical line-backend campaign, `REAL24` is the official real-instance anti-regression panel, `MR25` is the earlier 525-cell matched-representation matrix, and `NF25` is the earlier 1,120-cell natural-kernel factorial ablation. Each prefix is unique and must appear in a reader-facing label before the source instance name. The matched-representation benchmark and the natural-workload ablation reported in the main text are the later campaigns `w4-matched-representation-20260802` and `w8-natural-factorial-20260803-local`, registered in `data/manifest.yaml` with receipts `data/frozen/matched_representation_freeze_manifest.json` and `data/frozen/natural_workload_analysis.json`; `MR25` and `NF25` are retained as historical diagnostics. The per-bundle `SHA256SUMS` files under `PaperDraft/research/` seal each bundle at its freeze; five of them no longer match the current files (generator scripts and, for the claim-evidence bundle, its rows), which were revised after the freeze, and the seals are kept unchanged as historical records.

Every campaign manifest records or explicitly marks unavailable the backend topology, native gate set, seed, timeout policy, memory cap, tool versions, artifact/freeze commit, bridge sequence, certificate policy, immutable config hash, and frozen dataset hash. Historical campaigns did not enforce a memory cap or independent certificate; their manifests say `not_enforced_historical` and "diagnostic only" rather than imputing a modern policy. In particular, `HW24` uses 120 s for QPE/QAOA and 240 s for Grover, whereas `RP23` uses 600 s. Those rows may share an MQT source-instance string but cannot share a short label or be averaged.

The configuration hierarchy is single-parent: a campaign inherits only its named base profile and then supplies explicit overrides. Backend and config hashes use canonical JSON. The frozen source snapshots `frozen_hardware_aware_rows.json` and `frozen_real_panel_rows.json` live beside the manifests, so later changes to a runner cannot silently change a displayed row.

### S13.7 Claim–Evidence Matrix

Every headline statement in the abstract, introduction, resource section, and conclusion is registered with one primary theorem, proposition, table, figure, or artifact test and an explicit scope (Table S15). The machine-readable file `research/claim_evidence/claim_evidence.json` records the claim set of the earlier full-length version and is kept unchanged as a historical record.

## S14 Supplementary Experiments

In this section, rows labeled "baseline UCC" are frozen-control rows: they denote the upstream UCC v0.4.12 pre-semantic default path defined in the provenance map of Section S13, not a separate semantic method.

### S14.1 Exact-Semantics Witness Data

The following tables contain the full uniform-600 s comparison data for the exact-semantics witnesses discussed in Definition S11.6 of the main text.

| Requested | Actual | semantic UCC | `qiskit opt3` | PyZX `full_reduce` | TKET PauliSimp (rebased) | staq rotation folding | TKET GuidedPauliSimp |
|---|---|---|---|---|---|---|---|
| 4,000 | 3,998 | 42/24/12 | 9588/5593/4788 | 107/59/31 | 52/29/19 | 9188/4996/4788 | 13574/6392/4788 |
| 10,000 | 9,998 | 42/24/12 | 23988/13993/11988 | timeout | 62/36/28 | 22988/12496/11988 | 33974/15992/11988 |
| 20,000 | 19,998 | 42/24/12 | 47988/27993/23988 | timeout | 59/27/20 | 45988/24996/23988 | 67974/31992/23988 |
| 50,000 | 49,998 | 42/24/12 | 119988/69993/59988 | timeout | 73/44/35 | 114988/62496/59988 | 169974/79992/59988 |
| 100,000 | 99,998 | 42/24/12 | 239988/139993/119988 | timeout | 66/37/28 | 229988/124996/119988 | 339974/159992/119988 |

Table S16: Uniform-600 s comparison on the rational CP finite-range witness. Completed cells report gates/depth/cx; other entries are status outcomes and are not scored as structural wins. The family is certified only for $R \leq T = 131040$, with per-transducer tail constants. No external row is a formal theorem instance.

| Requested | Actual | semantic UCC | `qiskit opt3` | PyZX `full_reduce` | TKET PauliSimp (rebased) | staq rotation folding | TKET GuidedPauliSimp |
|---|---|---|---|---|---|---|---|
| 4,000 | 3,998 | 27/18/6 | 4799/2801/2394 | 38/19/8 | 36/14/6 | 3402/2201/2394 | 8786/4000/2394 |
| 10,000 | 9,998 | 27/18/6 | 12000/7001/5994 | 48/33/12 | 35/15/8 | 8502/5501/5994 | 21986/10000/5994 |
| 20,000 | 19,998 | 27/18/6 | 24000/14001/11994 | correctness failure | 38/16/8 | 17002/11001/11994 | 43986/20000/11994 |
| 50,000 | 49,998 | 27/18/6 | 60000/35001/29994 | timeout | 36/15/8 | 42502/27501/29994 | 109986/50000/29994 |
| 100,000 | 99,998 | 27/18/6 | 120000/70001/59994 | timeout | 37/16/8 | 85002/55001/59994 | 219986/100000/59994 |

Table S17: Uniform-600 s comparison on the irrational independent-Pauli witness. Completed cells report gates/depth/cx; other entries are status outcomes and are not scored as structural wins. The exact rank and irrational-angle certificate, rather than this numerical table, instantiate Definition S11.6.

## S14.2 Fourier Witness Diagnostics

The tables in this subsection support the main-text Fourier witness without serving as additional theorem instances. The canonical theorem-linked CP witness is the finite-range case in Table S16; the generalized topology/angle rows are robustness diagnostics, and the external stress rows use the budgets recorded in their frozen source files.

| Topology | Angles | Points | `qiskit` ok/TO | Semantic completed | Semantic output signatures |
|---|---|---|---|---|---|
| chain CP | 3 | 12 | 9 / 3 | all 12 | 23–27 gates, 17–18 depth, 6 `cx` |
| ring CP | 3 | 12 | 9 / 3 | all 12 | 32 gates, 23 depth, 8 `cx` |
| sparse CP 0.5 | 3 | 12 | 6 / 6 | all 12 | 25–27 gates, 11–12 depth, 6 `cx` |
| full-pair CP | 2 | 6 | 4 / 2 | all 6 | 42 gates, 24 depth, 12 `cx` |
| Total points | – | 42 | 28 / 14 | all rows | repetition-independent at fixed topology/angle |

Table S18: Generalized fixed-width Fourier witness suite. Each point is a paired run of semantic UCC and `qiskit opt3` on an $HD^rH$ circuit at $n = 4$, with requested sizes up to 50k gates. The rows are filtered from the recorded 284-row generalized suite by the $n = 4$ topology/angle/size slice. Recorded `qiskit opt3` timeout cells in that source use legacy 60 s or 90 s budgets, so the table is a robustness diagnostic rather than a uniform-600 s comparison.

## S14.3 Controlled Fixed-Basis Results

### S14.3.1 Repeated $\mathrm{QFT} + \mathrm{QFT}^{-1}$ (100k gates)

### S14.3.2 Repeated $\mathrm{QFT} + \mathrm{QFT}$ control (100k gates)

These finite controls show that, for this configured family and the tested pipelines, a large structural opportunity remains after controlling for the final target basis. They are not a separate theorem-level witness.

### S14.3.3 Systematic $\mathrm{QFT} + \mathrm{QFT}^{-1}$ external scaling

The inverse-QFT scaling control is not the primary theorem witness, because `qiskit opt3` also finds the zero-gate circuit at smaller sizes. Its role is to test scalability when both paths return the same zero-gate output. As shown in Table S25, the semantic artifact returns the zero-gate circuit at all tested sizes, is faster than `qiskit opt3` wherever `qiskit opt3` completes, and avoids the `qiskit opt3` timeouts at 50k and 100k. As an additional control, it also matches `qiskit_commutative_inverse` in output quality and is faster at every tested size.

| Width ($n$) | Checks | Output Gates | Depth | CX Count | Seed Variance |
|---|---|---|---|---|---|
| 4 | 15 / 15 | 42 | 24 | 12 | 0 |
| 5 | 15 / 15 | 65 | 32 | 20 | 0 |
| 6 | 15 / 15 | 93 | 40 | 30 | 0 |

Table S19: Correctness and seed-robustness certificate for the full-pair Fourier witness. The correctness experiment checks 3 widths $\times$ 3 requested sizes $\times$ 5 angle seeds using Qiskit `Operator.equiv`; all 45 configurations are equivalent to the original circuit up to global phase. The seed-robustness experiment contains 90 rows including `qiskit opt3`; semantic UCC has zero output gate-count variance across the five tested nonresonant angle seeds at each width.

| Method | Requested | Actual | Status | Gates/depth/`cx` | Certificate |
|---|---|---|---|---|---|
| semantic UCC (Fourier-layer IR enabled) | 100,000 | 99,998 | completed | 42/24/12 | `Operator.equiv=True` |
| phase-polynomial reference | 100,000 | 99,998 | completed | 42/24/12 | `Operator.equiv=True` |

Table S20: Large-instance correctness extension on the canonical $n = 4$, $r = 9999$ Fourier witness. The check is a finite-instance Qiskit `Operator.equiv` certificate for the two completed compressed outputs only; it does not certify the generalized topology/angle suite or prove an algebraic identity for all $r$.

## S14.4 Why Simple Pass Reordering Is Not Enough

To test whether the observed gains are mainly explained by a missing pass or pass-ordering issue inside `UCCDefaults`, we ran a controlled comparison between baseline UCC, several minimal `UCCDefaults`-style alternatives, and the semantic artifact. The minimal alternatives were `defaults_preinverse_only`, `defaults_reordered_only`, and `defaults_minimal_bundle`; the all-to-all cases were normalized to the same final target basis $B = [\texttt{cx}, \texttt{rx}, \texttt{ry}, \texttt{rz}, \texttt{h}]$, and the backend-aware cases used fixed transpiler seed 12345. This is a separate reorder-probe experiment, so its runtime numbers should be read independently from the frozen mainline tables below. Its hardware rows have reader labels `RP23-qpeexact-s12345-c89463b96` and `RP23-qaoa-s12345-c89463b96`; the canonical hardware campaign below instead uses the `HW24-...-c0f32747f` labels. The differing labels are different full experiment keys, not repeated measurements of one row.

Two qualifications are not visible in Table S26 itself. On `RP23-qaoa-s12345-c89463b96` the minimal bundle already matches the semantic artifact, so the reorder-only explanation is adequate there. On `RP23-qpeexact-s12345-c89463b96` it is not: the minimal bundle gives 1,893 gates, depth 576, `cx` 1,311, against 1,593, depth 420, `cx` 772 for the semantic artifact.

## S14.5 Public Benchmark Suite Results

To move beyond official library examples, we added two public benchmark sources: `MQT Bench` and `SupermarQ`.

**MQT Bench.** Representative results are:

- `mqt_qpeexact_32`: `qiskit opt3` = 1,891 gates, baseline UCC = 5,494, semantic UCC = 1,891;
- `mqt_qpeinexact_24`: `qiskit opt3` = 1,206 gates, baseline UCC = 3,682, semantic UCC = 1,206;
- `mqt_ae_8`: `qiskit opt3` = 207 gates, baseline UCC = 542, semantic UCC = 207;
- `mqt_draper_qft_adder_32`: `qiskit opt3` = 1,630 gates, baseline UCC = 5,278, semantic UCC = 1,630;
- `mqt_qaoa_32`: `qiskit opt3` = 1,422 gates, baseline UCC = 6,310, semantic UCC = 1,422;
- `mqt_grover_20`: `qiskit opt3` = 3,848,810 gates in 38.043 s, baseline UCC times out at $> 120$ s, and semantic UCC reaches the same output quality in 39.123 s.

| Width | Requested | semantic UCC | `qiskit opt3` | PyZX bridge | TKET FullPeephole |
|---|---|---|---|---|---|
| 5 | 4,000 | 65 | 11,711 | 14,640 | 30,369 |
| 5 | 10,000 | 65 | 29,315 | 36,640 | timeout |
| 6 | 4,000 | 93 | 12,108 | 15,321 | 33,881 |
| 6 | 10,000 | 93 | 30,417 | 38,487 | timeout |

Table S21: Representative external stress tests on the width-axis full-pair Fourier witness. Entries report output gates after the configured bridge pipeline and final target-basis compilation. PyZX completes all four runs but does not recover the bounded semantic forms 65 and 93; TKET completes the 4k runs with larger outputs and times out at 10k. The recorded budget in this stress file is 300 s.

| Topology | semantic UCC | `qiskit opt3` | PyZX bridge | TKET FullPeephole |
|---|---|---|---|---|
| chain CP | 27 | 6,851 / 17,136 | 10,838 / 27,121 | 20,555 / timeout |
| ring CP | 32 | 7,996 / 19,996 | 11,984 / 29,984 | 23,987 / 59,987 |
| sparse CP 0.5 | 27 | 6,851 / 17,136 | 10,838 / 27,121 | timeout / timeout |
| full-pair CP | 42 | 10,783 / 26,983 | 13,574 / 33,974 | timeout / timeout |

Table S22: Topology-axis external stress test at $n = 4$ with nonresonant seed 0. Each external-baseline cell reports output gates at requested sizes 4k / 10k. The deterministic sparse-CP draw uses the same number of CP edges as chain CP and is graph-isomorphic to a chain at $n = 4$, which explains the matching `qiskit opt3` and PyZX gate totals for those two rows. The recorded budget in this stress file is 300 s.

Thus, on `MQT Bench`, the semantic artifact repeatedly recovers `qiskit opt3`-level output quality and clearly improves over baseline UCC. The added `qpeinexact`, `ae`, and `draper_qft_adder` cases are especially relevant because they extend the phase-ladder/Fourier-layer story beyond one exact-QPE benchmark into inexact QPE, amplitude estimation, and QFT-based arithmetic. Runtime still remains weaker on the hardest Grover-style case.

Additional mirrored/conjugation positive cases make the mirrored-shell heuristic more concrete. On the official `grover_real` instance, baseline UCC returns 287,047 gates while the semantic artifact and `qiskit opt3` both return 79,029. On `MQT Bench` Grover instances, the same pattern persists across scale: `mqt_grover_8` gives 18,992 → 5,170, `mqt_grover_12` gives 259,797 → 71,706, and `mqt_grover_16` gives 2,112,285 → 581,098, where in each case the semantic artifact records the same gate count as `qiskit opt3` while clearly improving on baseline UCC. The large fixed-basis mirrored Grover benchmark shows the same parity pattern: baseline UCC gives 4,122,006 gates, while the semantic artifact and `qiskit opt3` both give 1,158,011. These cases make the mirrored/conjugation line less dependent on the routed timeout-recovery interpretation of `hw_mqt_grover_20`.

A dedicated mirrored/conjugation scaling sweep makes this support more systematic. On public `MQT Bench` Grover instances, the semantic artifact records the same gate count as `qiskit opt3` at sizes 8, 12, and 16, reducing baseline UCC from 18,992 to 5,170 gates, from 259,797 to 71,706 gates, and from 2,112,285 to 581,098 gates, respectively. On the fixed-basis `grover_mirrored` repeated-shell scaling sweep, the same parity pattern holds at 10k, 20k, 50k, and 100k input gates: baseline UCC gives 412,206, 824,406, 2,061,006, and 4,122,006 output gates, while the semantic artifact and `qiskit opt3` give 115,811, 231,611, 579,011, and 1,158,011 gates. Thus this second structural family targeted by the artifact has empirical positive cases in both public Grover scaling and large repeated-shell fixed-basis scaling.

**SupermarQ.** Representative results are:

- `supermarq_mermin_bell_8`: baseline UCC = 386 gates, depth 135; semantic UCC = 76 gates, depth 44; `qiskit opt3` = 76 gates, depth 44;
- `supermarq_qaoa_vanilla_12`: baseline UCC = 947 gates, depth 225; semantic UCC = 222 gates, depth 80; `qiskit opt3` = 222 gates, depth 80;
- `supermarq_hamiltonian_sim_8`: baseline UCC = 71 gates, depth 36; semantic UCC = 51 gates, depth 24; `qiskit opt3` = 51 gates, depth 24.

This second public benchmark source provides additional finite anti-regression checks on nontrivial

| Pipeline | Representation tested | Observed behavior in the disclosed suites | Interpretation |
|---|---|---|---|
| semantic UCC (Fourier-layer IR enabled) | pre-basis Fourier-layer term | completed 5/5; 42 gates, depth 24, 12 `cx` at every tested scale | semantic side |
| phase-polynomial reference | same Qiskit input; only commuting-diagonal coefficient aggregation | completed 5/5; 42 gates, depth 24, 12 `cx` at 4k–100k | authors' protocol-specific constructive control |
| `qiskit opt3` | preset target-basis pipeline | completed 5/5; grows from 9,588 gates at 4k to 239,988 gates at 100k | strong completed baseline, but not bounded |
| PyZX configured bridge | Qiskit–QASM–PyZX basic optimization–Qiskit | completed 4/5; grows through 169,974 gates at 50k and times out at 100k | configured bridge does not recover bounded form |
| PyZX `full_reduce` | QASM to PyZX graph; `full_reduce`; extract and lower | rational witness: completes only at 4k with 107 gates, depth 59, 31 `cx`; timeout at 10k–100k. Irrational witness: completes and validates at 4k and 10k with 38 and 48 gates; correctness failure at 20k; timeout at 50k and 100k | strong small-scale simplification; correctness failure remains distinct from timeout and error |
| TKET FullPeephole | direct Qiskit–TKET bridge to FullPeepholeOptimise | timeout on all five canonical sizes under the 600 s budget | no completed bounded recovery in this protocol |
| TKET PauliSimp (unrebased configuration) | direct Qiskit–TKET bridge to PauliSimp | predicate error on all five sizes for both witness families | provenance-only configuration; not a structural loss |
| TKET PauliSimp (rebased) | generic DecomposeBoxes, AutoRebase to {CX,Rz,Rx}, RemoveRedundancies, PauliSimp, RemoveRedundancies | completes and validates at all five scales: 52–73 gates on the rational witness and 35–38 on the irrational witness | generic aggregate reconstruction crosses the formal boundary |
| staq rotation folding | generic target-basis QASM2 through public `fold_rotations` | completes and validates at all five scales, but grows from 9,188 to 229,988 gates on the rational witness and from 3,402 to 85,002 on the irrational witness | completed public baseline; growing output is not an impossibility result |
| TKET GuidedPauliSimp | direct Qiskit–TKET bridge to GuidedPauliSimp | completed 5/5 on each witness; returns 13,574 to 339,974 gates on the rational witness, exactly the gate count of its lowered input, and 8,786 to 219,986 on the irrational witness | no optimization on flat input: the pass acts only inside `CircBox` regions |
| artifact UCC (Fourier-layer IR disabled) | same artifact with Fourier-layer semantic path disabled | in ablation/resource runs, completes at 4k, 10k, and 20k with 9,588, 23,988, and 47,988 gates; times out at 50k and 100k | semantic-path causality check; completed rows materialize large outputs |

Table S23: External-pipeline status matrix and constructive controls for the rational CP and irrational independent-Pauli witnesses. Completed outputs and status outcomes are separated. The rebased and unrebased PauliSimp configurations are distinct, and no row is treated as a formal transducer instance.

| | $\mathrm{QFT} + \mathrm{QFT}^{-1}$ | | $\mathrm{QFT} + \mathrm{QFT}$ | |
|---|---|---|---|---|
| Method | Output Gates | Output Depth | Output Gates | Output Depth |
| translation only | 400,000 | 142,500 | 400,000 | 140,000 |
| baseline UCC | 968,784 | 276,266 | 1,207,504 | 317,501 |
| semantic UCC | 0 | 0 | 347,500 | 125,000 |

Table S24: Controlled fixed-basis results for the repeated inverse and non-inverse QFT controls at 100k input gates.

families and also exposes a limitation: the semantic artifact is not uniformly better on every benchmark family.

## S14.6 Hardware-Aware Results

We added a backend-aware comparison using a fixed 20-qubit bidirectional line backend with native operations `u`, `sx`, `p`, `cx`, `measure`, and `id`. The benchmark source is `MQT Bench`. The reader-facing rows are `HW24-qpeexact-s12345-c0f32747f`, `HW24-qaoa-s12345-c0f32747f`, and `HW24-grover-s12345-c0f32747f`; the full source instance remains part of each registry primary key.

| Input gates | `qiskit opt3` | | `qiskit_commutative_inverse` | | semantic UCC |
|---|---|---|---|---|---|
| | Gates | Runtime | Gates | Runtime | Runtime |
| 4,000 | 0 | 1.052 s | 0 | 0.082 s | 0.063 s |
| 10,000 | 0 | 6.090 s | 0 | 0.182 s | 0.124 s |
| 20,000 | 0 | 24.596 s | 0 | 0.341 s | 0.279 s |
| 50,000 | timeout | $> 90$ s | 0 | 0.979 s | 0.829 s |
| 100,000 | timeout | $> 90$ s | 0 | 2.271 s | 1.651 s |

Table S25: External scaling on repeated $\mathrm{QFT} + \mathrm{QFT}^{-1}$. The semantic UCC artifact returns zero gates at all tested sizes; the table reports runtime because output quality is identical to the zero-gate reference wherever the references complete. Timeout cells in this source use the recorded 90 s budget.

| Case | Method | Gates | Depth | CX count | Runtime |
|---|---|---|---|---|---|
| `phase_estimation_real` | baseline UCC | 471,819 | 328,271 | 58,491 | 75.538 s |
| | `defaults_preinverse_only` | 485,780 | 332,924 | 82,925 | 80.045 s |
| | `defaults_reordered_only` | 485,780 | 332,924 | 82,925 | 80.005 s |
| | `defaults_minimal_bundle` | 485,780 | 332,924 | 82,925 | 80.084 s |
| | semantic UCC | 166,805 | 119,204 | 58,491 | 1.989 s |
| | `qiskit opt3` | 166,805 | 119,204 | 58,491 | 1.254 s |
| `qaoa_ring_100k` | baseline UCC | 300,055 | 163,055 | 40,000 | 2.431 s |
| | `defaults_preinverse_only` | 434,104 | 217,962 | 40,000 | 11.352 s |
| | `defaults_reordered_only` | 445,216 | 217,962 | 40,000 | 10.120 s |
| | `defaults_minimal_bundle` | 434,104 | 217,962 | 40,000 | 10.818 s |
| | semantic UCC | 100,000 | 62,000 | 40,000 | 3.131 s |
| | `qiskit opt3` | 100,000 | 62,000 | 40,000 | 0.307 s |
| `RP23-qpeexact s12345-c89463b96` | baseline UCC | 2,077 | 626 | 1,262 | 0.126 s |
| | `defaults_preinverse_only` | 1,893 | 576 | 1,311 | 0.292 s |
| | `defaults_reordered_only` | 1,900 | 582 | 1,312 | 0.289 s |
| | `defaults_minimal_bundle` | 1,893 | 576 | 1,311 | 0.287 s |
| | semantic UCC | 1,593 | 420 | 772 | 0.296 s |
| | `qiskit opt3` | 1,572 | 407 | 812 | 0.033 s |
| `RP23-qaoa s12345-c89463b96` | baseline UCC | 2,088 | 522 | 1,439 | 0.098 s |
| | `defaults_preinverse_only` | 2,050 | 487 | 1,458 | 0.270 s |
| | `defaults_reordered_only` | 2,050 | 487 | 1,458 | 0.269 s |
| | `defaults_minimal_bundle` | 2,050 | 487 | 1,458 | 0.271 s |
| | semantic UCC | 2,050 | 487 | 1,458 | 0.278 s |
| | `qiskit opt3` | 2,091 | 532 | 1,530 | 0.048 s |

Table S26: Controlled comparison of baseline UCC, minimal `UCCDefaults`-style alternatives, the semantic artifact, and `qiskit opt3`. Small default-pipeline fixes explain part of the improvement on some cases, but they do not account for the main gains on the real and structured workloads. The overlapping `phase_estimation_real` rows use the per-method output `cx` counts from the recorded real-instance data. The keyed hardware rows are generated from the frozen registry; their `RP23` labels distinguish this reorder-probe configuration from the `HW24` campaign below.

**(a) `HW24-qpeexact-s12345-c0f32747f`**

| Method | Output Gates | Output Depth | CX Count | Runtime |
|---|---|---|---|---|
| `qiskit opt3` | 1,572 | 407 | 812 | 0.056 s |
| baseline UCC | 1,976 | 648 | 1,350 | 0.174 s |
| semantic UCC | 1,572 | 407 | 812 | 0.235 s |

**(b) `HW24-qaoa-s12345-c0f32747f`**

| Method | Output Gates | Output Depth | CX Count | Runtime |
|---|---|---|---|---|
| `qiskit opt3` | 2,091 | 532 | 1,530 | 0.078 s |
| baseline UCC | 2,057 | 513 | 1,503 | 0.141 s |
| semantic UCC | 2,050 | 487 | 1,458 | 0.112 s |

Table S27: Canonical hardware-aware results for the QPE and QAOA reader labels, generated from the frozen keyed dataset.

**`HW24-qpeexact-s12345-c0f32747f.`** This is a clean fixed-seed parity case: the semantic artifact matches `qiskit opt3` in total gates, depth, and `cx` count, while still remaining much better than baseline UCC on all structural metrics. The tradeoff is runtime rather than output quality.

**`HW24-qaoa-s12345-c0f32747f.`** This is the clearest hardware-aware external-baseline improvement in the paper: the semantic artifact improves on `qiskit opt3` in total gates, depth, and `cx` count, while also improving on baseline UCC in total gates, depth, and `cx` count.

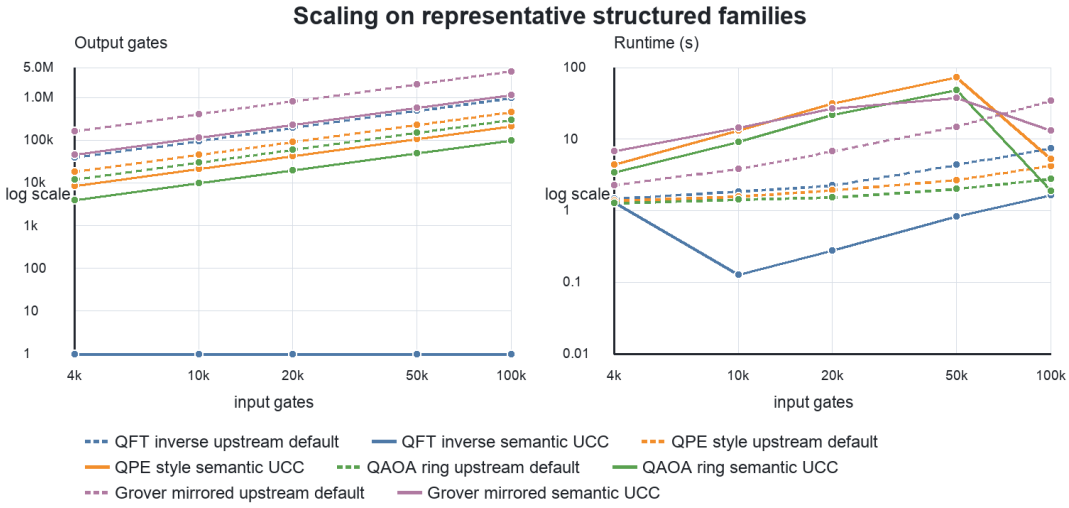



Figure S11: Scaling behavior on representative structured families. Left: output gate count versus input gate count for upstream default (baseline UCC) and the semantic UCC artifact. Right: runtime versus input gate count. The semantic path preserves clear quality advantages across scale, while runtime behavior remains family-dependent.

`HW24-grover-s12345-c0f32747f.` `qiskit opt3` and baseline UCC still time out under this setting, but the semantic artifact returns a result in 1.999 s via a stronger mirrored/self-inverse repeated-run backend fallback. The resulting circuit is still extremely large, but the selector lowers total gates, depth, and `cx` simultaneously (to 6,467,857 total gates, depth 4,094,196, and `cx` count 3,045,048), so we treat this as a robustness result rather than a main competitive routed-quality claim.

## S14.7 Stability and Seed Sensitivity

We measured repeated-run stability on the frozen artifact in two ways:

1. five repeated runs for the official real instances and the canonical hardware-aware seed,
2. a five-seed sweep for the backend-aware `MQT Bench` cases.

   The main conclusions are:

- the official real-instance outputs are deterministic across repeated runs,
- the hardware-aware outputs are deterministic for each fixed seed,
- `hw_mqt_qaoa_20` remains better than `qiskit opt3` in total gates, depth, and `cx` count across all five tested seeds 0, 1, 42, 12345, and 54321,
- `hw_mqt_qpeexact_20` is now a fixed-seed parity case rather than a strict improvement.

Thus, the hardware-aware claim is a stable workload-family result across a small fixed seed portfolio, rather than a single favorable seed.

## S14.8 Scaling and Ablation

The main scaling trends are summarized in Fig. S11. The size sweep covers 4k, 10k, 20k, 50k, and 100k inputs for QFT-inverse, QPE-style, QAOA-ring, and mirrored-Grover families; the QAOA-ring block uses 20 qubits. The most important observations are stable quality behavior across scale and improving repeated-family runtime once the semantic representation and dispatch controller are enabled. The method is most effective on exact or highly repeated structure: `qft_inverse`

remains maximally strong across all tested sizes, `qpe_style` remains a strong positive fixed-basis case, `qaoa_ring` consistently avoids UCC inflation, and the mirrored-Grover family continues to match Qiskit quality at 100k while remaining more expensive. Direct spot checks on the semantic artifact give `qpe_style` = 82,200 gates at 50k in 2.504 s and 164,400 gates at 100k in 4.636 s, reflecting the phase-ladder plus repeated-run controller improvements.

The non-monotonic semantic runtime between the 20k and 50k points is a dispatch effect visible in the implementation, not an inferred hardware effect. The 20,000-gate full-preset limit `_MAX_FULL_PRESET_REFERENCE_SIZE` allows the 19,998-gate input to enter the full preset-reference comparison, whereas the 49,998-gate input exceeds that threshold and uses the cheaper repeated-prefix/hierarchical route. The corresponding canonical witness runtimes are 28.314 s and 20.934 s. This threshold explains the local drop; it is not evidence of asymptotically decreasing compilation time.

The ablation picture is narrow. The dominant practical component is conservative candidate control on recoverability-sensitive families, most clearly on the 10,000-input-gate `qaoa_ring` row, where the full configuration returns 10,000 gates while the no-candidate-selection variant returns 30,055. By contrast, `qft_inverse`, `qft_control`, `qpe_style`, and `grover_mirrored` are less sensitive to the removal of individual structural rules. This reinforces the paper's current framing: the main contribution is a bounded semantic representation and dispatch controller that preserve recoverability before flat materialization, rather than a single local identity.

## S15 Reproducibility

For scale, the headline campaigns comprise 592 traced reference-compiler runs; a 1,080-cell matched-representation design and a separate 1,728-cell external campaign containing TKET PauliSimp and PyZX full-reduce, with completed metrics accepted only when supported by the stated symbolic or dense certificate; 288 complete fixed-total-error fault-tolerant estimates under a common algorithmic, synthesis, and logical-failure budget; and 21,060 natural-workload and negative-control cells crossing a six-factor ablation, in which 5 structured families satisfy the predeclared all-size/all-seed criterion and the two negative controls show 0 systematic regressions. Explicitly labelled historical diagnostics retain their original artifact provenance.

The canonical registry is `data/manifest.yaml`; the normalized schema is `schema/experiment.schema.json`. Its complete key is

```
experiment_id, instance_id, representation_id,
method_id, seed, backend_hash, tool_version,
artifact_commit, config_hash.
```

The normalized Parquet index contains 24,748 unique records and retains 48 predicate-error rows outside quality means. The data audit checks CSV/Parquet mirrors, conflicting numeric cells, duplicate TeX labels, source hashes, row counts, orphaned frozen files, and historical versus canonical campaign disposition. It finds no duplicate primary keys, registered-file omissions, conflicting mirror cells, hash drift, or unregistered frozen artifacts among the committed canonical sources.

All measurement macros, tables, and figures used by the W3–W9 sections are generated from registered frozen sources by `scripts/build_all_tables.py` and `scripts/build_all_figures.py`; the operation and unique source row for each printed number are recorded in `data/provenance_map.yaml` and tested by `tests/test_paper_number_provenance.py`. The correctness language, symbolic checker, and mutation/dense audit are in `certificates/`; the QRE schemas and two physical profiles are in `qre/`. The approximately 700-MB raw W3–W8 run tree, including manifests, logs, certificates, seeds, and package records, is versioned in the separate private `OIerYangJZ/paper1-data` archive at the revision recorded in `data/manifest.yaml`; its absence is reported but does not invalidate the committed frozen-data rebuild.

The one-command entry point is `reproducibility/run_submission.sh`. From a fresh clone it creates the locked project environment when needed, reruns frozen-data audits, regenerates all registered tables and figures, executes the full test suite, builds the canonical, arXiv, Quantum-submission, and supplement PDFs, and rejects unresolved references. This procedure reproduces the submitted analysis from immutable frozen datasets; it does not rerun the complete measurement campaigns. The clean-room runner copies a minimal artifact to an independent temporary

directory, creates a fresh environment bound to the locked project packages, reruns 9 headline-theorem tests, compiles and densely certifies a matched QAOA/Ising instance through the authors and TKET paths, and exercises 17 deterministic synthesis pairs in a fixed-total-error QRE subset. This is working-directory isolation, not a claim of host-binary independence. Complete audit and limitation records are `DATA_AUDIT_REPORT.md`, `CLEAN_ROOM_REPORT.md`, `QRE_REPORT.md`, and `NATURAL_WORKLOAD_REPORT.md`.

## References


[1] Leon Riesebos, Xiang Fu, Savvas Varsamopoulos, Carmen G. Almudever, and Koen Bertels. "Pauli frames for quantum computer architectures". In Proceedings of the 54th Annual Design Automation Conference (DAC 2017). Pages 76:1–76:6. (2017).

[2] Emanuel Knill. "Quantum computing with realistically noisy devices". Nature **434**, 39–44 (2005).

[3] George Watkins, Hoang Minh Nguyen, Keelan Watkins, Steven Pearce, Hoi-Kwan Lau, and Alexandru Paler. "A high performance compiler for very large scale surface code computations". Quantum **8**, 1354 (2024). arXiv:2302.02459.

[4] Ali Javadi-Abhari, Matthew Treinish, Kevin Krsulich, Christopher J. Wood, Jake Lishman, Julien Gacon, Simon Martiel, Paul D. Nation, Lev S. Bishop, Andrew W. Cross, Blake R. Johnson, and Jay M. Gambetta. "Quantum computing with Qiskit" (2024). arXiv:2405.08810.

[5] Seyon Sivarajah, Silas Dilkes, Alexander Cowtan, Will Simmons, Alec Edgington, and Ross Duncan. "t|ket⟩: A retargetable compiler for NISQ devices". Quantum Science and Technology **6**, 014003 (2021).

[6] Alexander Cowtan, Silas Dilkes, Ross Duncan, Will Simmons, and Seyon Sivarajah. "Phase gadget synthesis for shallow circuits". In Proceedings 16th International Conference on Quantum Physics and Logic (QPL 2019). Volume 318 of Electronic Proceedings in Theoretical Computer Science, pages 213–228. (2020). arXiv:1906.01734.

[7] Aleks Kissinger and John van de Wetering. "PyZX: Large scale automated diagrammatic reasoning". In Proceedings 16th International Conference on Quantum Physics and Logic (QPL 2019). Volume 318 of Electronic Proceedings in Theoretical Computer Science, pages 229–241. (2020).